



UNIVERSIDAD DE TALCA
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA CIVIL EN COMPUTACIÓN

**Aplicación web para el apoyo del proceso de
fabricación de muebles a medida**

ANGEL NICOLÁS BRAVO MUÑOZ

Profesor Guía: FRANCISCO ARIAS

Memoria para optar al título de
Ingeniero Civil en Computación

Curicó – Chile
SEPTIEMBRE 2025



UNIVERSIDAD DE TALCA
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA CIVIL EN COMPUTACIÓN

**Aplicación web para el apoyo del proceso de
fabricación de muebles a medida**

ANGEL NICOLÁS BRAVO MUÑOZ

Profesor Guía: FRANCISCO ARIAS

Profesor Informante: PROFESOR INFORMANTE 1

Profesor Informante: PROFESOR INFORMANTE 2

Memoria para optar al título de
Ingeniero Civil en Computación

El presente documento fue calificado con nota: _____

Curicó – Chile

SEPTIEMBRE 2025

Dedicado a ...

AGRADECIMIENTOS

Agradecimientos a ...

TABLA DE CONTENIDOS

página

ÍNDICE DE FIGURAS

página

ÍNDICE DE TABLAS

página

1. Introducción

En las pequeñas provincias y zonas rurales de Chile, el proceso de diseño y fabricación de muebles suele depender de métodos tradicionales que dificultan la estimación precisa de costos. La falta de herramientas digitales accesibles limita la capacidad de los artesanos y pequeños fabricantes para planificar, optimizar y competir en mercados más exigentes.

Desarrollar una plataforma web que permita diseñar muebles en 2D (simulando a un dibujo en un cuaderno), estimar costos automáticamente y optimizar el uso de materiales, todo desde una interfaz sencilla y accesible. La solución está orientada a mejorar la productividad de usuarios en contextos rurales, facilitando la transición hacia procesos más eficientes y digitalizados.

1.1. Diseño de la propuesta

1.1.1. Contexto

El rubro de mueblería abarca la industria dedicada al diseño, fabricación, comercialización y distribución de muebles para diversos espacios como hogares, oficinas y más. Los principales materiales utilizados para su fabricación son madera, metal, plástico, vidrio, cuero, telas o combinaciones de estos [?].

Además se distinguen 2 procesos de fabricación de muebles:

- **Fabricación estándar en serie:** Los cuales se basan en la producción masiva con medidas y diseños estandarizados, como por ejemplo IKEA [?].

- **Fabricación de mueble a medida o artesanal:** Se enfoca en crear diseños únicos adaptados a las necesidades específicas del cliente. [?]

Según el estudio de Rodrigo Vidal 2017 [?] sobre la fabricación y comercialización de muebles multifuncionales en Chile, más del 95 % de los encuestados manifestó la necesidad de optimizar el espacio en sus viviendas, lo que representa una oportunidad para el rubro de fabricación de muebles a medida en comparación con los muebles pre-fabricados de medidas estándar. Adicionalmente, un 55,43 % considera que el mobiliario actual no cumple con estas necesidades. Además, los atributos más valorados por los consumidores son el estilo y diseño, la multifuncionalidad y la exclusividad.

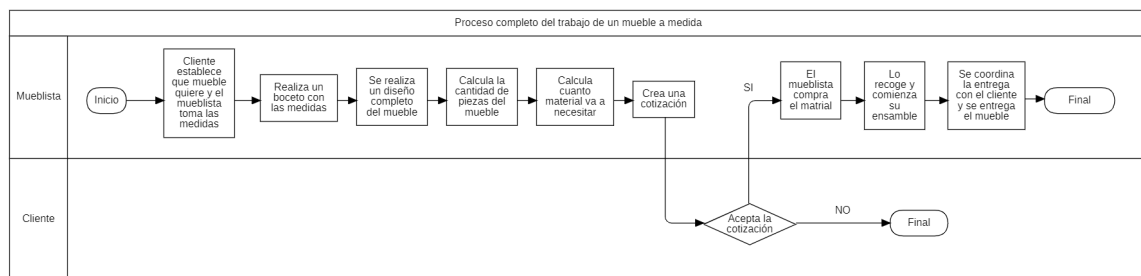
En un estudio de 'Informes de Expertos' realizado en 2024, se evidencia que el sector mueblero en Chile es parte de las actividades económicas de relevancia creciente, dado que es una demanda en aumento por la clase media que busca personalizar sus espacios eligiendo muebles de acuerdo a sus preferencias de funcionalidad y estilo, con un mercado estimado en USD 660 millones en 2023 y una proyección de crecimiento anual de 4,8 % entre 2025 y 2034, y así alcanzar un valor de 1,006.82 millones de USD en 2034. [?]

A nivel regional, los parámetros son diferentes, ya que en la Región Metropolitana las ventas de muebles registraron una caída acumulada de 2,6 % en los primeros meses de 2025 [?] mientras que en regiones como Valparaíso y La Araucanía se observó un crecimiento positivo del 5,9 % durante el mismo período [?].

Este proyecto se centra en el proceso de fabricación de muebles a medida, el cual se detalla a continuación.

La naturaleza de este oficio nace al traspasar las técnicas de diseñar y crear muebles de generación en generación, al no ser un tipo de enseñanza formal, cada mueblista debe buscar la manera de poder diseñar o plantear sus ideas, por una parte se encuentran aquellos que usan lápiz y papel, generando un lento proceso de los cálculos y dimensiones para hacer una entrega de un presupuesto de material.

El flujo de interacción más común en este rubro es contactar al mueblista, entre ambas partes acordar una hora en común para la visita de medición, que es la instancia en la que el cliente propone lo que necesita y requiere para que el mueblista plantee el diseño de un mueble que cumpla con los requerimientos, lo que muchas veces suele ser un boceto rápido para que el cliente tenga una idea del producto que obtendrá. Una vez planteada la idea con las medidas, el mueblista procede a realizar el diseño más técnico para calcular la cantidad de material que va a usar con sus correspondientes medidas. Dependiendo de las dimensiones y complejidad del mueble este proceso de diseño técnico puede tardar entre 2 y 8 horas. A continuación el mueblista solicita al proveedor de materiales una cotización. Dicha suma de dinero es comunicada y solicitada al cliente para que el mueblista inicie el proceso de fabricación. Si el cliente acepta la cotización se procede a comprar los materiales en los proveedores indicados. Este proceso puede demorar entre 1 y 3 días hábiles. Una vez recibido el material se comienza el armado del mueble el cual puede demorarse desde un par de días hasta semanas, dependiendo de las piezas y la complejidad del mismo. Para cuando esté listo se coordina con el cliente para realizar la entrega del mueble en el lugar solicitado por el cliente. La Figura 1 describe el proceso de fabricación de muebles a medida anteriormente explicado.



Esta información respalda la pertinencia de desarrollar una plataforma digital que permita al mueblista diseñar muebles a medida en un entorno 2D, visualizando en tiempo real el impacto estético y funcional de sus decisiones, permitiendo además generar una cotización ajustada a materiales, dimensiones y acabados seleccionados.

1.1.2. Planteamiento del problema

El proceso de fabricación de muebles a medida es generalmente transmitido de generación en generación. La oportunidad de mejora o modernización es lenta o muy básica, ya que hay mueblistas que trabajan con bocetos en papel, requieren de mucha precisión en las escalas al momento del dibujo, lo que en muchos casos se ve afectado por las limitaciones educativas presentes en zonas rurales [?]. Si se considera un mueblista con experiencia en la confección de diseños, el tiempo que invierte al ser todo manual puede tomar entre X y Z horas, ya que se debe confeccionar el diseño considerando que posteriormente se debe estimar la cantidad de cortess (segmentos del material a utilizar) que son requeridos según el tamaño de la “plancha” (tamaño original o máximo del material en cuestión), por lo cual para lograr un presupuesto completo al detalle es un proceso que requiere dedicación y exactitud.

En el mercado existen aplicaciones generales y específicas que eventualmente podrían ser adoptadas por los fabricantes de muebles a medida, pero factores como el costo, la curva de aprendizaje o incluso la tradición del rubro, dan cuenta de la ausencia de una solución que se adapte a sus procesos y que no sean ellos los que deban adaptar sus procesos a la aplicación [?]. Además, en general por los tiempos involucrados en los procesos de cotizaciones iniciales muchas veces no se puede competir con las grandes tiendas, que al tener un stock o que solo requieran de la entrega, genera mucha diferencia comparativamente en los tiempos de respuesta [?].

1.1.3. Propuesta de solución

La solución propuesta consiste en el diseño y desarrollo de una plataforma web para el diseño 2D que permita al mueblista mantener su esquema de trabajo tradicional ahora en un entorno digital, aprovechando además de funcionalidades tecnológicas que buscan apoyarlo en dicho proceso, orientada a apoyar y optimizar el tiempo en el rubro del mueblista o carpintero artesanal. Este sistema considera un espacio de diseño de muebles el cual disponga de los elementos comunes del proceso de fabricación de muebles a medida, por ejemplo espacios modulares de cajoneras, espacios de repisas, colgadores, cubiertas, entre otros. Donde el mueblista pueda escoger cada módulo que necesite, simplificando su usabilidad teniendo que sólo definir las dimensiones (alto, ancho y profundidad) del mueble de acuerdo con los requerimientos del usuario, sin necesidad de ensamblar cada pieza de forma individual. Una vez definido el diseño, el sistema debe calcular automáticamente la cantidad de materiales necesarios para su fabricación (por ejemplo, el número de planchas

de melamina requeridas) y generar un listado detallado de materiales.

A continuación se describen las etapas involucradas en la elaboración de una cotización para la fabricación de un mueble, junto con sus tiempos estimados y propuestas de optimización:

- **Etapas de diseño:** El mueblista realiza un boceto con las medidas de alto, ancho y profundidad del mueble solicitado. Este proceso puede tardar entre 2 a 4 horas, dependiendo de la complejidad del diseño y la cantidad de piezas involucradas. Se propone optimizar esta etapa mediante el uso de la aplicación web, permitiendo realizar el diseño de forma más rápida en el entorno digital. Se espera una reducción del tiempo de diseño de un 25 %, logrando tiempos entre 1 hora y 45 minutos a 3 horas.
- **Etapas de cubicación:** A partir del diseño, se calcula la cantidad de cortes necesarios de material requerido. Este proceso actualmente se realiza de forma manual y puede tardar entre 2 a 8 horas, dependiendo de la cantidad de piezas y cortes. Con la aplicación web, se busca que la cubicación se realice automáticamente al finalizar el diseño, reduciendo este tiempo a solo minutos, lo que representa una mejora de aproximadamente 90 %.
- **Tiempo total de cotización:** Promediando ambas etapas, el tiempo para entregar una cotización es de aproximadamente 9 horas (3 horas en diseño + 6 horas en cubicación). Con la implementación de la solución propuesta, este tiempo se reduciría significativamente, permitiendo entregar cotizaciones en menos de 4 horas.
- **Impacto en el proceso completo:** Considerando que la fabricación de un mueble puede tardar entre 2 a 10 días, la optimización de las etapas de diseño y cubicación permitiría una mejora promedio del proceso un 15 %.

Además se busca contar con una base de datos del historial de muebles para que el mueblista, al momento del diseño, pueda escoger un mueble ya fabricado y modificar las medidas del mueble en general o modificar los módulos según corresponda.

Una vez realizado el cálculo de material, se es posible modificar los detalles de la cotización para que el cliente vea el cambio del valor total dependiendo del color, grosor o composición del material.

A su vez la aplicación considera la implementación de un módulo de estadística básica que permitirá registrar y visualizar los periodos del año con mayor demanda de muebles.

Este módulo ofrecerá indicadores como el máximo, promedio y mínimo de ventas, segmentados por mes o por año.

1.2. Objetivos

1.2.1. Objetivo General

Optimizar el proceso de diseño y cubicación en la fabricación de muebles a medida, mediante el desarrollo de una aplicación web intuitiva que sirva como herramienta para la creación de diseños personalizados, el cálculo de materiales y la generación de cotizaciones precisas.

1.2.2. Objetivos específicos

1. Implementar una aplicación web que provea de un espacio de diseño en dos dimensiones (2D) orientado a muebles para que el fabricante pueda generar diseños personalizados en base a los requisitos del cliente.
2. Implementar el módulo de cubicación que apoye al fabricante en el cálculo de materiales y costos, agilizando el proceso de propuesta para el cliente.
3. Desarrollar un repositorio de muebles que permita guardar, reutilizar y modificar diseños previamente realizados.
4. Implementar una biblioteca de materiales, colores, grosores y accesorios que permita y facilite la personalización de los diseños.
5. Automatizar la generación y envío de cotizaciones detalladas e interactivas para que el cliente o mueblista elija entre un abanico de posibilidades.
6. Reducir el tiempo en el proceso de fabricación de muebles a medida en un 15 % respecto al método manual.
7. Implementar un módulo de estadísticas que permita al mueblista visualizar datos relacionados a demanda e ingresos.
8. Evaluar la satisfacción y usabilidad de la aplicación a través de una encuesta de funcionalidad y usabilidad.

1.3. Alcances

- La plataforma se diseña para ser utilizada en entorno web responsive.
- No se considera la implementación de una aplicación nativa móvil debido a que nativa móvil, debido a que módulos como el de cotizaciones para generar el presupuesto está más pensado para trabajarse en un computador antes que en un teléfono.
- Los requerimientos se capturan a partir de entrevistas con un mueblista artesanal de la localidad de Teno.
- La plataforma no considera el diseño de muebles en 3D dado que se busca que el mueblista mantenga su esquema de trabajo tradicional, sin agregar complejidad al trabajo en pantallas reducidas.
- Se considera la implementación de un área de diseño 2D especializada para muebles modulares.
- La plataforma no considera envíos físicos de materiales ni coordinación de logística de entrega.
- La aplicación no considera gestión de clientes, debido a que está solo orientada a un tipo de usuario.
- Requerirá que los usuarios tengan conexión a Internet para acceder a la plataforma.
- La plataforma no considera la implementación de métodos de pagos ni gestión de transacciones financieras.

1.4. Plan de Trabajo

A continuación se presenta el plan de trabajo del proyecto.

1.4.1. Descripción de etapas del proyecto

En esta sección se presentan las etapas del proyecto, tareas más importantes y la cantidad de semanas de duración que se considera realizar en cada etapa.

1. Requerimientos (1 semana)

- Capturar requerimientos mediante entrevistas.

2. Planificación (2 semanas)

- Creación y estimación de historias de usuario.
- Definición de tecnologías.
- Diseño de modelo de base de datos, mockups y arquitecturas.

3. Implementación (19 semanas)

- Configuración inicial de proyecto.
- Implementar módulo de diseño de muebles.
- Implementar módulo de visualización de cortes.
- Implementar módulo de gestión de cotización.
- Implementar módulo de muebles realizados.
- Implementar módulo de biblioteca de materiales.
- Integración de los módulos desarrollados.
- Implementar módulo estadística.

4. Evaluación (2 semanas):

- Aplicar pruebas de caja negra.
- Correcciones de aplicación.
- Aplicar prueba de usabilidad con su encuesta.

5. Conclusión (2 semanas):

- Comparar métricas de entrevista inicial y final.
- Realizar conclusiones y completar documento.

Se estima un total de 26 semanas para el desarrollo del proyecto dejando una semana para una revisión exhaustiva del documento, además de ir trabajando en la implementación iteración a iteración. Para mayor detalle de la planificación ver la carta Gantt en el anexo A.

1.4.2. Tareas para objetivos específicos

Para cumplir los objetivos específicos se tiene en cuenta las siguientes acciones para cada uno de ellos:

- OE1: Implementar una aplicación web que provea de un espacio de diseño en dos dimensiones (2D) orientado a muebles para que el fabricante pueda generar diseños personalizados en base a los requisitos del cliente
 1. Etapa 3: Implementar módulo de diseño de muebles.
 2. Etapa 3: Implementar módulo de gestión de cotización.
- OE2: Implementar el módulo de cubicación que apoye al fabricante en el cálculo de materiales y costos, agilizando el proceso de propuesta para el cliente.
 1. Etapa 3: Implementar módulo de visualización de cortes.
- OE3: Desarrollar un repositorio de muebles que permita guardar, reutilizar y modificar diseños previamente realizados.
 1. Etapa 3: Implementar módulo de muebles realizados.
- OE4: Implementar una biblioteca de materiales, colores, grosores y accesorios que permita y facilite la personalización de los diseños.
 1. Etapa 3: Implementar módulo de biblioteca de materiales.
- OE5: Automatizar la generación y envío de cotizaciones detalladas e interactivas para que el cliente o mueblista elija entre un abanico de posibilidades.
 1. Etapa 3: Implementar módulo gestión de cotizaciones.
- OE6: Optimizar el tiempo en el proceso de fabricación de muebles a medida en un 15 % respecto al método manual.
 1. Etapa 3: Implementar módulo de diseño de muebles.
 2. Etapa 3: Implementar módulo de visualización de cortes.
 3. Etapa 3: Implementar módulo de gestión de cotización.
 4. Etapa 3: Implementar módulo de muebles realizados.
 5. Etapa 3: Implementar módulo de biblioteca de materiales.
- OE7: Implementar un módulo de estadísticas que permita al mueblista visualizar datos relacionados a demanda e ingresos.
 1. Etapa 3: Implementar módulo estadística.

- OE8: Evaluar la satisfacción y usabilidad de la aplicación a través de una encuesta de funcionalidad y usabilidad.
 1. Etapa 4: Aplicar pruebas de caja negra.
 2. Etapa 4: Aplicar prueba de usabilidad con su encuesta.

1.5. Resumen del capítulo

En este capítulo se presentó el contexto del proyecto, enfocado en la optimización del proceso de diseño y cálculos de la fabricación de muebles a medida. Además, se definió la problemática principal que está asociada al método manual para la estimación de costos y cubicación. Para lo cual se propone como solución una aplicación web que permita el diseño 2D y los cálculos automáticos, así reduciendo los tiempos de entrega y potenciales errores.

En el siguiente capítulo se abordarán conceptos básicos del proyecto como tecnologías a utilizar, estado del arte y metodologías de desarrollo y evaluación

2. Marco Teórico

Este capítulo está orientado a describir conceptos necesarios para entender el foco del proyecto. Además, se presentan las tecnologías que son parte del proyecto.

2.1. Conceptos globales

- **Cubicación:** es un término fundamental en arquitectura, ingeniería y construcción. Es el proceso técnico de medir y calcular las cantidades de materiales, mano de obra y elementos que se necesitarán para ejecutar un proyecto u obra. Teniendo como objetivo principal cuantificar el proyecto para elaborar un presupuesto preciso, planificar la compra de materiales y programar los trabajos.
- **Optimización de Corte:** Concepto que a menudo se confunde con la cubicación, este es el proceso algorítmico para determinar la disposición óptima de un conjunto de piezas (cortes) sobre un material estándar (como una plancha de melamina) para minimizar el desperdicio de material.
- **Módulo de Mueble:** Se refiere a una unidad o sección pre-diseñada y estandarizada que forma parte de un mueble más grande (Por ejemplo, un módulo de cajonera, un módulo de repisa).

2.2. Conceptos de software

- **Frontend:** Es la capa de la aplicación con la que el usuario interactúa directamente, osea, corresponde a todo lo visual de la aplicación web desde la vista del usuario.
- **Backend:** Es la capa lógica que se ejecuta en el servidor, invisible para el usuario. Se encarga de procesar las peticiones del frontend, aplicar la lógica de negocio (ej. calcular la cubicación, guardar diseños) e interactuar con la base de datos.

- Canvas: Es un elemento de HTML que actúa como un 'lienzo' en blanco, permitiendo dibujar gráficos, animaciones y manipular imágenes de forma dinámica usando JavaScript. Es la tecnología base sobre la cual funciona la librería Konva.js [?] para crear el espacio de diseño 2D.
- Servidor: Es el componente que espera, recibe y procesa las solicitudes que envía el cliente, centralizando el procesamiento de datos y ejecutando los algoritmos o funciones. Además es quien interactúa directamente con la base de datos para entregar la respuesta al cliente.
- Cliente: Se refiere a la persona que le da uso a aplicación, mediante un dispositivo (teléfono, computador, tablet, entre otros) inicia la comunicación solicitando los servicio o recursos que entrega la aplicación.
- Petición HTTP: Es el mensaje que el cliente envía al servidor para solicitar una acción específica sobre un servicio o recurso. Además, cada petición incluye un método que indica la intención de la acción, siendo estos los más comunes:
 - GET: Se usa para solicitar datos.
 - POST: Se usa para enviar datos nuevos.
 - PUT: Se usa para actualizar datos existentes.
 - DELETE: Se usa para eliminar datos.

2.3. Tecnologías

2.3.1. React

Para el frontend que representa la interfaz visual en donde el usuario interactuará con el sistema se evaluaron 3 frameworks: React [?], Vue [?] y Angular [?].

- React [?] es un framework mantenido por Meta y la comunidad, utilizado para construir interfaces de usuario (UI) interactivas y reutilizables. Su enfoque se basa en el concepto de 'componentes', que son partes o piezas de código encapsuladas e independientes que manejan su propio estado.
- Vue [?] es un framework que cuenta con un enfoque progresivo, siendo creado y mantenido por la comunidad, destacando por su baja curva de aprendizaje y su sintaxis de plantillas basada en HTML. Su flexibilidad le permite ser un núcleo pequeño al

que se le pueden añadir módulos oficiales para enrutamiento y gestión de estado a medida que el proyecto crece.

- Angular [?] es un framework mantenido por google y a diferencia de los anteriores que son construidos en Javascript o compatibles, angular es creado sobre TypeScript. Además, sigue una arquitectura Modelo-Vista-Controlador. Otra gran diferencia es la biblioteca ya que angular es una solución 'Todo en uno' que incluye su propio sistema de enrutamiento.

Por su flexibilidad y compatibilidad, React [?] ha sido seleccionado como el framework ideal para la implementación de la solución propuesta.

Cuadro 2.1: Comparativa de frameworks web

Característica	React	Angular	Vue.js
Tipo	Biblioteca UI	Framework completo (MVC)	Framework progresivo
Curva de Aprendizaje	Media	Alta	Baja
Ecosistema	Enorme, requiere bibliotecas de terceros	Integral (todo incluido)	Flexible, núcleo pequeño
Integración con Canvas	Excelente. Librerías como react-konva facilitan la integración.	Posible, pero más verboso y complejo de integrar.	Buena integración, similar a React.
Soporte y Comunidad	Muy alta (mantenido por Meta)	Alta (mantenido por Google)	Alta (mantenida por la comunidad)
Rendimiento (DOM)	Alto (Virtual DOM)	Bueno (DOM Real)	Alto (Virtual DOM)

2.3.2. MongoDB

- SQL es el lenguaje estándar utilizado para gestionar y consultar base de datos relacionales. Este modelo de base de datos organiza los datos en tablas compuestas por filas y columnas con un esquema rígido y definido.

- NoSQL se refiere a la categoría de gestores de bases de datos que difieren del modelo relacional de SQL. Estos sistemas se diseñaron para ofrecer mayor escalabilidad y un esquema flexible.

Es por la flexibilidad y escalabilidad de la propuesta es que usar una base de datos NoSQL es la mejor opción. Pero dentro de todas las opciones destacan Firebase [?] y MongoDB [?].

- Firebase [?] es una plataforma de desarrollo propiedad de Google para el desarrollo de aplicaciones web y móviles. Siendo más que una simple base de datos se considera un Backend-as-a-Service(BaaS) que ofrece un conjunto de herramientas y servicios gestionados en la nube para que el desarrollador pueda centrarse en la experiencia del usuario.
- MongoDB [?] es un sistema gestor de base de datos NoSQL orientado a documentos. Almacena los documentos flexibles similares a JSON denominados BSON, lo que permite manejar datos semiestructurados de forma natural. Este modelo de esquema flexible es útil para casos de uso donde las estructuras de datos son jerárquicas o que evolucionan con el tiempo, ya que permite guardar la información en una sola entidad en lugar de dividirla en múltiples tablas.

Es por esta flexibilidad, experiencia y forma de manipular documentos, en el esquema que permite guardar, reutilizar y modificar diseños más eficiente y escalable que intentar normalizar esa información en múltiples tablas relacionales, es que MongoDB [?] es la mejor opción para este proyecto.

Cuadro 2.2: Comparativa de gestor de base de datos

Características	MongoDB (NoSQL)	Bases de Datos Relacionales (SQL)
Modelo de Datos	Orientado a Documentos.	Orientado a Tablas.
Esquema	Flexible. Cada documento puede tener una estructura única.	Rígido. Todas las filas de una tabla deben seguir la misma estructura.
Manejo de Datos	Natural. Un diseño de mueble completo (con sus módulos, piezas y materiales) puede guardarse en un único documento, preservando su estructura compleja.	Complejo. Requeriría múltiples tablas (ej: Muebles, Modulos, Piezas, Materiales) y consultas complejas para 'reconstruir' un solo diseño.
Aplicable	Proyectos con datos no estructurados o semi-estructurados.	Proyectos con datos altamente estructurados.

2.3.3. Konva.js

Es una librería de Javascript que extiende las capacidades del 'Canvas' 2D de HTML. Permitiendo crear gráficos 2D interactivos y de alto rendimiento, trabajando con las formas (rectángulos, círculos, etc.) como 'objetos' en un 'escenario' (Stage) y 'capas' (Layers). Facilitando la gestión de eventos (arrastrar, soltar), animaciones y transformaciones de objetos.

Es por estos motivos que Konva.js [?] es de las mejores opciones para usar en comparación de Fabric.js que fue otra opción a utilizar.

Cuadro 2.3: Comparativa de librerías para el diseño en lienzo

Características	Konva.js	Fabric.js
Enfoque Principal	Lienzo 2D de alto rendimiento con capas (escenario, capas, formas).	Editor de gráficos 2D interactivo y manipulación de objetos.
Rendimiento	Optimizado para animaciones y miles de formas.	Bueno, pero puede ser más pesado al manejar muchos objetos.
Modelo de Capas	Explícito (Arquitectura de Stage - Layer - Shape).	Basado en un lienzo (canvas) con una colección de objetos.
Compatibilidad con React	Excelente. Tiene un wrapper dedicado (react-konva) muy popular.	Buena, pero la integración es más manual.
Manejo de Eventos	Robusto y granular (eventos por forma, capa y escenario).	Robusto, centrado en la manipulación de objetos (redimensionar, rotar).

2.3.4. Trello

Trello es una aplicación de propiedad de Atlassian que se utiliza en la gestión de proyectos visual y colaborativa. Su interfaz se basa en el método Kanban, empleado en tableros, listas y tarjetas para organizar tareas y flujos de trabajo.

En comparación a otras herramientas similares como Jira o Asana, Trello presenta una mejor curva de aprendizaje, por lo que esta es la alternativa seleccionada.

2.3.5. Email.js

Entre las herramientas para el envío de correo automatizado destacan principalmente 2 que son Email.js y Sendgrid, los cuales describiremos a continuación:

- Email.js es un servicio de plataforma que permite enviar correos electrónicos directamente desde las aplicaciones sin necesidad de configurar o mantener un servidor propio. Su funcionamiento se basa en actuar de intermediario seguro. En donde el desarrollador crea plantillas predefinidas dentro del servicio de Email.js

- Sengrid es una plataforma de comunicación como servicio basada en la nube, diseñada para la entrega de correos a gran escala. Esta plataforma gestiona una compleja infraestructura de envío, la reputación de las direcciones IP y la escalabilidad, con el objetivo de maximizar la tasa de entrega evitando que estos correos sean marcados como spam.

Por simplicidad y funcionalidades del plan gratuito, se selecciona Email.js ya que da 2 template y 200 correos mensuales en el plan gratis para usar con cualquier cuenta de correo y viendo las dimensiones y requisitos del proyecto no veo que se requiera más.

2.3.6. Node.js

Dentro del entrono del backen de la propuesta tenemos muchas opciones para que trabajen la lógica y las interacciones que ocurren detrás de la interfaz del usuario. Entre las opciones tenemos Node.js [?] con JavaScript, Spring Boot con Java, Laravel con PHP, Flask con python, entre otros. Estas tecnologías permiten relizar las mismas funciones, pero lo que hace la diferencia es la sintaxis de cada lenguaje.

Teniendo en cuenta lo anterior y que la propuesta se basa en trabajar con JavaScript la mejor elección es Node.js [?] con Express por su compatibilidad con la propuesta.

2.3.7. Docker

Docker [?] es una plataforma que ofrece el servicio de contenedores para desarrolladores mediante imágenes de software para poder construir un programa en un espacio controlado y poder traspasarlo a otro dispositivo sin correr con errores de versiones en las instalaciones que suelen dar problemas. Es por eso que se utiliza Docker al ser un contenedor se puede crear un progama con sus versiones correspondientes y compartirlo sin correr riesgos de que pueda fallar.

2.3.8. Gloogle Cloud

Es una plataforma que ofrece servicios en la nube para desarrollar y alojar aplicaciones. Entre sus servicios destacan el alojamiento web, bases de datos, almacenamiento y análisis de datos.

2.4. Estado del arte

En cuanto al estado del arte del proyecto, existen distintos tipos de herramientas tanto de pago como gratuitas, a continuación, se mencionan y describen algunas herramientas reconocidas que estan relacionadas con el proyecto.

1. **AutoCAD** [?]. Es un software de diseño de paga que se utiliza para dibujar, diseñar y modelar en 2D y 3D de forma precisa con sólidos, superficies, objetos de malla, características de documentación, etc. Incluye características para automatizar tareas y aumentar la productividad, como la comparación de dibujos, el recuento, la adición de objetos y la creación de tablas.

2. **IKEA PLanner** [?]. Es un software web de IKEA que se encarga de dar ideas según el catálogo de IKEA, en donde se tiene un plano 3D de un espacio genérico de cocina, baño, etc. el cual permite agregar muebles de IKEA para ajustarlo según las necesidades del cliente.

3. **Cabinet Vision** [?]. Es un software de HEXAGON el cual se utiliza para el diseño de muebles 2D y 3D incluyendo el 'listado de cortes', automatizando la exportación a diferentes formatos para las solicitudes de material. Esta aplicación solo es de uso empresarial.

4. **Moblo** [?]. Es un software que se utiliza para crear diseños 3D de un mueble según las necesidades a partir de figuras simples, permitiendo modificar su color o tipo de material, de uso gratuito.

5. **Cut list** [?]. Es un software que se utiliza para ubicar la lista de cortes de los muebles en la plancha de manera visual y así saber cuanto material se va a usar en el ensamble del mueble. Se ingresan todos los valores de manera manual y también lo que se pierde por corte. Además permite exportar lo listado en PDF o PNG.

6. **SketchUp** [?]. Es un software que se utiliza para modelado 2D y 3D de estructuras, ya sea construcción, muebles o diseño de interiores, siendo de paga para profesionales, con posibilidad de vista en realidad virtual (VR).

7. **Opticutter** [?]. Es un software web que se usa de calculadora de hoja de corte en línea la cual obtiene una solución utilizando el algoritmo de optimización adecuado.

Cuadro 2.4: Comparativa de software mencionado

Función	Propuesta	1	2	3	4	5	6	7
Diseño 2D/3D	2D	2D y 3D	3D	2D y 3D	3D	No	Sí	No
Cubicación Automática	Sí	No	No	Sí	Sí	Sí	No	Sí
Generación Cotización	Sí	No	No	No	No	No	No	No
Curva de Aprendizaje	Baja	Alta	Baja	Alta	Media	Baja	Alta	Baja
Accesibilidad	Web	Nativa	Web	Nativa	Nativa	Web	Nativa	Web

2.5. Metodologías

2.5.1. Metodología de desarrollo

Dado que este proyecto será desarrollado de manera individual, se realiza una comparación entre diferentes metodologías ágiles destacando tres opciones: Kanban [?], Personal Extreme Programming (PXP) [?] y Lean Startup [?], todas son independientes del lenguaje de programación o framework seleccionado. Estas metodologías comparten el enfoque ágil.

Personal Extreme Programming (PXP): Adaptación individual de la metodología Extreme Programming (XP). PXP usa prácticas como desarrollo incremental, pruebas automatizadas, diseño simple y refactorización continua, pero ajustadas a un entorno uni-personal.

Cuadro 2.5: Comparativa de metodologías de desarrollo

Característica	PXP	Kanban	Lean Startup
Enfoque Principal	Calidad del código y desarrollo incremental para un solo desarrollador.	Flujo de trabajo visual y continuo.	Validación de hipótesis de mercado/producto.
Funcionamiento	Basado en prácticas de ingeniería.	Basado en un tablero (Lista 'To Do').	Se basa en un ciclo de feedback con el cliente/mercado.
Flexibilidad	Alta. Los cambios se gestionan en iteraciones cortas.	Alta. Los cambios pueden entrar al flujo en cualquier momento.	Muy Alta. El objetivo es cambiar basado en los datos aprendidos.
Disciplina Técnica	Alta. Exige prácticas de ingeniería de software.	Baja. No se enfoca en prácticas de código, solo de flujo.	Baja. El foco no es la calidad técnica, sino la velocidad para aprender.
Aplicabilidad	Alta. Se enfoca en la ejecución de un proyecto con requisitos definidos y alta calidad técnica.	Alta. Útil para gestionar tareas, pero no guía la calidad del software.	Medio. Prioriza validar un producto en el mercado.

Para este proyecto se elige Personal Extreme Programming (PXP) dado que se adapta a las características requeridas en el contexto. Además PXP se centra en la calidad del código desde el inicio, además de que permite avanzar en ciclos cortos, lo que facilita ajustes constantes y mejora continua proporcionando flexibilidad durante el desarrollo.

Para el desarrollo de esta propuesta se seguirán distintas fases que contempla la metodología:

Requerimientos: Durante esta etapa se captan los requisitos del proyecto en conjunto con el cliente. Estos requerimientos se convierten en la fase siguiente en historias de usuario, que permiten enfocar el desarrollo en la experiencia del usuario.

Planificación: Durante esta fase se desarrolla la creación y estimación de las historias de usuarios, la elección de tecnologías a utilizar, diseño de mockups y arquitectura.

Iteración: Durante esta etapa se indica la inicialización de cada iteración o ciclo, la cual puede durar entre 1 y 4 semanas.

Diseño: Durante esta fase se desarrolla el diseño lógico de módulos y funciones correspondientes a cada iteración.

Implementación: Durante esta fase se realiza el desarrollo mediante programación de las historias de usuario asociadas a una tarea, para salir de esta fase debe funcionar sin errores y pasar todas las pruebas unitarias, en caso de no ser así se debe refactorizar el código realizado.

Sistema de Pruebas: Durante esta fase se realiza el testeado para validar las funcionalidades desarrolladas durante la iteración y se debe verificar si la solución implementada con los requisitos iniciales del proyecto.

Retrospectiva: Esta fase demarca el final de cada iteración, se realiza un análisis de los datos recopilados durante las fases y de las propuestas de mejora. Esta etapa puede marcar el inicio de una nueva iteración o el final del desarrollo del proyecto.

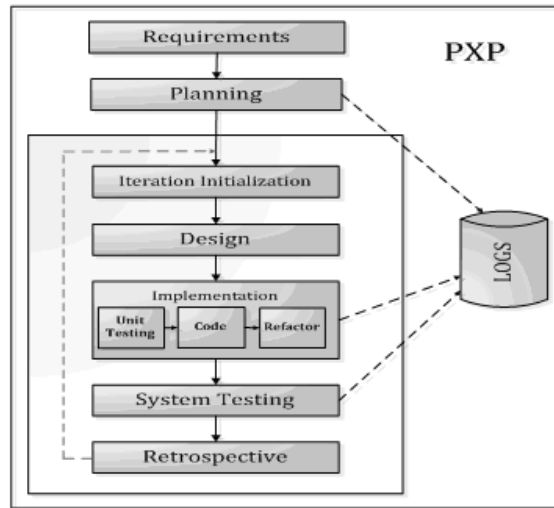


Figura 2.1: Diagrama de las fases de metodología Personal Extreme Programming (PXP) [?]

2.5.2. Metodología de evaluación

Prueba de caja negra

Las pruebas de caja negra se centran en evaluar el comportamiento funcional de la aplicación desde la perspectiva del usuario, sin considerar la estructura interna del código ni los algoritmos utilizados. Este enfoque permite validar que el sistema responde correctamente a los datos de entrada y cumple con los requisitos establecidos en la documentación técnica. Para ello, se diseñan casos de prueba basados en las especificaciones funcionales, verificando que las salidas obtenidas coincidan con las esperadas. [?]

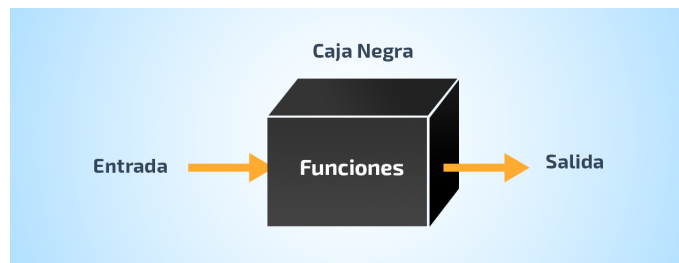


Figura 2.2: Proceso de Prueba de Caja Negra

Prueba de usabilidad

Las pruebas de usabilidad permiten validar la experiencia del usuario final con la aplicación desarrollada. Para este propósito, se aplicará la técnica System Usability Scale (SUS), una herramienta creada por John Brooke en 1986 y ampliamente utilizada en la industria del software. Esta prueba consiste en un cuestionario de 10 afirmaciones evaluadas mediante la Escala de Likert, donde los participantes expresan su nivel de acuerdo. A partir de estas respuestas, se calcula un puntaje total entre 0 y 100 que refleja la percepción de usabilidad: valores inferiores a 50 indican una experiencia inaceptable, entre 50 y 68 se consideran deficientes o mediocres, y puntuaciones superiores a 68 reflejan una aceptación adecuada del sistema por parte del usuario. [?]

2.6. Resumen del capítulo

En este capítulo se describieron los conceptos necesarios para entender el proyecto, además se describieron las tecnologías a utilizar, comparando opciones y justificando las elecciones. También se abordó el estado del arte, en donde se presentaron aplicaciones que ofrecen funcionalidades similares a la propuesta. Finalmente se detalló la metodología de desarrollo y evaluación, justificando sus elecciones.

En el siguiente capítulo se abordará la aplicación de la metodología elegida, junto a las adaptaciones necesarias para el proyecto y los resultados esperados de los objetivos específicos.

3. Metodologías

En este capítulo se describe como la metodología de desarrollo ya presentada se aplicará en la propuesta y los resultados esperados de los objetivos específicos.

3.1. Personal Extreme Programming (PXP)

Como se mencionó en el capítulo anterior, PXP al ser la versión individual de Extreme Programming (XP), se basa en el desarrollo personal de software manteniendo las fases claves como Requerimientos, Planificación, Iteración y Pruebas.

3.1.1. Requerimientos

Durante esta etapa donde el desarrollador realiza una entrevista con al cliente, para visualizar lo requerimientos del sistema. Estos requerimientos se convierten en la fase siguiente en historias de usuario, que permiten enfocar el desarrollo en la experiencia del usuario.

3.1.2. Planificación

Durante esta etapa la actividad central consiste en la creación, priorización y organización de las historias de usuario (HU). También se toman las decisiones de la composición de la aplicación, tales como las configuraciones de las tecnologías a utilizar, arquitecturas de software y mockups.

Cuadro 3.1: Clasificación de las historias de usuario

HU	Actor	Descripción
HU01	Mueblista	Como usuario quiero iniciar sesión en el sistema.
HU02	Mueblista	Como ususario quiero un canvas 2D para poder hacer diseños.
HU03	Mueblista	Como usuario quiero crear, guardar, cargar y eliminar el diseño de un mueble.
HU04	Mueblista	Como usuario quiero que el diseño se pueda hacer modular.
HU05	Mueblista	Como usuario quiero que las medidas de los módulos agregados puedan ser editables.
HU06	Mueblista	Como usuario quiero que los módulos sean móviles para acomodarlos según la necesidad.
HU07	Mueblista	Como usuario quiero que el sistema me calcule la cantidad de material.
HU08	Mueblista	Como usuario quiero que se muestre un listado de las piezas que se van a utilizar.
HU09	Mueblista	Como usuario quiero que el sistema pueda generar cotizaciones a partir del diseño.
HU10	Mueblista	Como usuario quiero que se vea una previsualización de la cotización.
HU11	Mueblista	Como usuario quiero que en la previsualización de la cotización se puedan personalizar.
HU12	Mueblista	Como usuario quiero que las cotizaciones se puedan exportar como pdf
HU13	Mueblista	Como usario quiero que el sistema permita enviar cotizaciones por correo
HU14	Mueblista	Como usuario quiero disponer de una biblioteca de materiales
HU15	Mueblista	Como ususario quiero poder agregar, editar y eliminar materiales de la biblioteca
HU16	Mueblista	Como usuario quiero ver estadíscas básicas
HU17	Mueblista	Como usuario quiero que se puedan filtrar las estadísticas por periodos

Para la clasificación de prioridad se utilizará la clasificación MuSCoW la cual es una técnica utilizada en la priorización de requerimientos en el desarrollo de software [?]. MuSCoW es el acrónimo derivado de Must have (Debe tener), Should have (Debería tener), Could have (Podría tener) y Won't have (No tendrá, por ahora). A continuación se describen las categorías:

- **Must Have:** Representa los objetivos esenciales o imprescindibles para que el sistema funcione correctamente. Sin estos elementos, el producto final sería inviable y no cumpliría con su propósito.
- **Should Have:** Estos son los requisitos importantes, pero no fundamentales para el funcionamiento del sistema. Si bien su ausencia no generará errores críticos, pero su implementación mejora la funcionalidad del sistema y se considera de alta prioridad en caso de que el tiempo y los recursos estén a favor.
- **Could Have:** Incluyen características adicionales que mejorarían la experiencia del usuario o la eficiencia del sistema. Se implementan solo si hay tiempo y recursos suficientes tras completar los requisitos de mayor prioridad.
- **Won't have:** Requisitos que se han identificado como no esenciales en esta etapa del desarrollo. No se implementarán en la versión actual del proyecto, aunque podrían considerarse en futuras versiones o iteraciones del sistema.

Para poder calcular el peso a las historias de usuario utilizaré la secuencia de Fibonacci ya que es una técnica de estimación del esfuerzo requerido para cada historia de usuario. El uso de esta escala no lineal obliga a diferenciar claramente entre tareas, evitando la falsa precisión de las estimaciones temporales en etapas tempranas del desarrollo. En esta escala permite categorizar las tareas de la siguiente manera:

- **Baja complejidad (1-3 pts.):** Tareas rutinarias, de bajo riesgo y con requisitos claros.
- **Media Complejidad (5 pts.):** Tareas que requieren lógica específica o integración de componentes.
- **Alta Complejidad (8-13 pts.):** Tareas con alta incertidumbre tecnológica, lógica algorítmica compleja o dependencias críticas.

A continuación, se presenta la clasificación MuSCoW para la prioridad de desarrollo y estimación de esfuerzo de las historias de usuario:

Cuadro 3.2: Clasificación MuSCoW y estimación de esfuerzo de las historias de usuario

HU	MuSCoW	Estimación de esfuerzo
HU01	Must	3
HU02	Must	5
HU03	Must	3
HU04	Must	8
HU05	Must	3
HU06	Must	3
HU07	Must	13
HU08	Must	3
HU09	Should	5
HU10	Should	2
HU11	Could	5
HU12	Should	2
HU13	Won't	3
HU14	Should	2
HU15	Should	3
HU16	Could	5
HU17	Could	2

Cuadro 3.3: Objetivos específicos con sus historias de usuario

Objetivos Específicos (resumidos)	Historias de Usuario (HU)
OE01: Implementar canvas 2D	HU02, HU04, HU05, HU06
OE02: Implementar el módulo de cubicación	HU07, HU08
OE03: Gestionar diseños realizados	HU03
OE04: Biblioteca de materiales	HU14, HU15
OE05: Automatizar cotizaciones	HU09, HU10, HU12, HU11, HU13
OE06: Reducir el tiempo del proceso de fabricación de muebles	Este objetivo es el resultado de OE01, OE02 y OE05. Se logra principalmente con HU02, HU04, HU07, HU09.
OE07: Implementar estadísticas	HU16, HU17
OE08: Evaluar la satisfacción y usabilidad	Aplicando encuesta SUS sobre las interfaces principalmente en: HU02, HU05, HU09 y HU014
Requisito General	HU01

Planificación historias de usuario

Cuadro 3.4: Planificación historias de usuario

Iteración	Historias de Usuario	Objetivo de la Iteración
Iteración 1	HU01	configuración del stack tecnológico e inicio de sesión
Iteración 2	HU02, HU03, HU04, HU05, HU06	Construir el diseñador 2D modular (guardar y cargar).
Iteración 3	HU07, HU08	Implementar el algoritmo de cubicación y mostrar la lista de piezas.
Iteración 4	HU09, HU10, HU12, HU14, HU15	Generar cotizaciones en PDF y su personalización.
Iteración 5	HU14, HU15	Gestionar biblioteca de materiales.
Iteración 6	HU16, HU17	Implementar estadísticas generales.
Iteración 7	HU13	Implementar notificaciones por correo.
Iteración 8	HU03	Implementar biblioteca de muebles

3.1.3. Iteración

Esta etapa es la de mayor duración e importancia en la metodología, la cual se divide en las siguientes etapas: Inicio de Iteración, Diseño, Implementación, Sistema de pruebas y Retrospectiva. A continuación, se describe la aplicación de cada una de las etapas.

Inicio de iteración

Durante esta etapa se crean las tareas específicas para cada historia de usuario correspondientes a cada iteración. Para llevar el control de las tareas se pueden usar diferentes herramientas mientras permitan una estructura sólida de tabla.

Iteración	HU	Tarea	Descripción	Estado
2	HU01	T01	Crear diagrama de secuencia	Completado
2	HU01	T02	Crear frontend del inicio de sesión	Completado
2	HU01	T03	Crear ruta para crear usuario	Completado
2	HU01	T04	Crear ruta para validar usuario	Completado

Figura 3.1: Extracto de la tabla de registro de tareas

Diseño

En esta etapa, se crean los diagramas de secuencia de las historias de usuario asociadas a la iteración. A continuación, se muestra un diagrama de secuencia de la función de login o inicio de sesión.

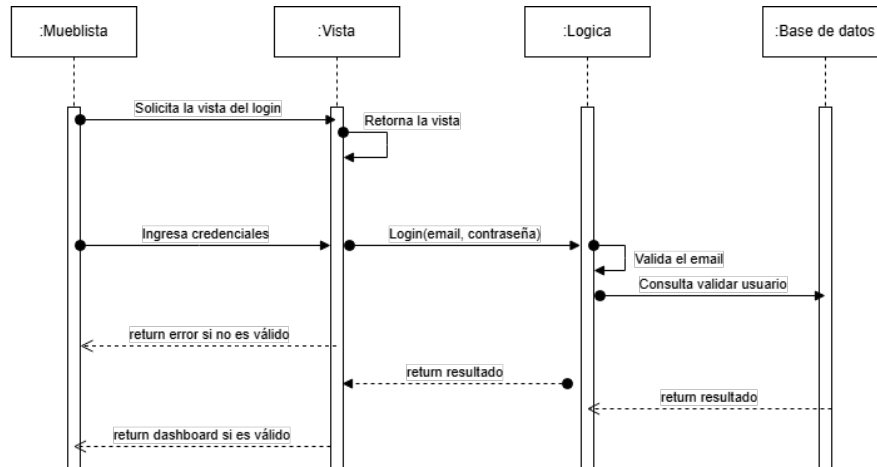


Figura 3.2: Diagrama de secuencia para inicio de sesión

Implementación

En esta etapa se lleva a cabo el desarrollo de código, pruebas unitarias y refactorización. La implementación sigue los diagramas de secuencia para guiar el flujo de las funcionalidades y los mockups propuestos. Si bien la metodología PXP recomienda el enfoque TDD o Test-Driven Development, que se basa en crear pruebas unitarias, se procede con código y finalmente con la refactorización, pero para este caso se ajustará el proceso a desarrollar el código, proceder con las pruebas unitarias y finalizar con refactorizar en caso de fallar alguna prueba.

Sistema de Pruebas

En esta etapa se crean y ejecutan las pruebas correspondientes a todas las tareas definidas en el Inicio de Iteración. En caso de fallar alguna prueba se debe volver a la fase de implementación para corregir el error, en caso de aprobar las pruebas se procede a la siguiente etapa. Para llevar el control se pueden usar diferentes herramientas que permitan listar para llevar una buena gestión de las pruebas hechas y tareas realizadas.

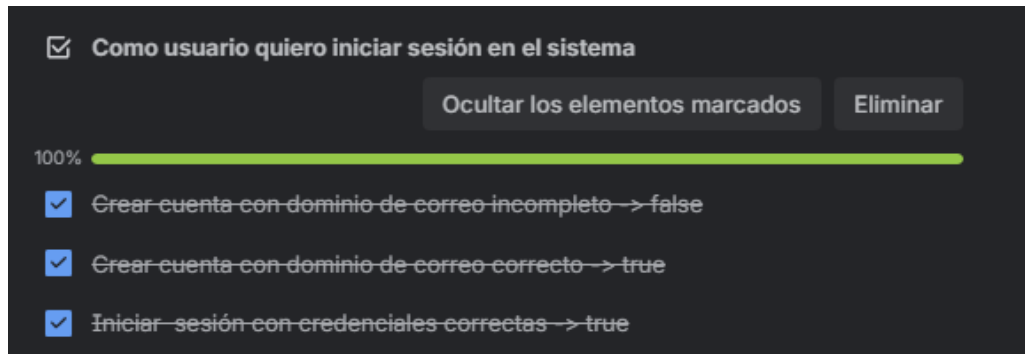


Figura 3.3: Extracto de lista de comprobación

Retrospectiva

Esta etapa marca el final de la iteración, en donde se deben verificar si el 'peso' estimado para las historias de usuario es igual al que tomó completarla, para evitar subestimar o sobreestimar tareas o historias futuras. También permite evaluar el avance de la propuesta, logrando identificar como se van cumpliendo con los plazos e identificar potenciales retrasos o problemas. Al finalizar esta etapa se tiene que decidir si se da inicio a una nueva iteración o finalizar las iteraciones y dar por completado el desarrollo de la aplicación.

3.2. Metodologías de evaluación

En esta sección se presentan las metodologías de evaluación a utilizar en el proyecto.

3.2.1. Prueba de caja negra

Las pruebas de caja negra es un método de evaluación de software, la cual se enfoca en las entradas y salidas del programa, sin la necesidad de tener conocimiento de la estructura interna del programa.

Para mantener el registro de las pruebas se pueden usar diferentes tipos de herramientas, en donde solo se necesita una estructura que contenga el identificador, la descripción de la prueba, las entradas, las salidas y el resultado esperado. A continuación, en la Figura 3.4 se ilustra una hoja de cálculo que contiene un ejemplo de la estructura de los registros de pruebas.

Identificador	Descripción	Entrada	Salida	Resultado esperado
1	Descripción	Entrada1	Salida1	Resultado esperado
2	Descripción	Entrada2	Salida2	Resultado esperado
3	Descripción	Entrada3	Salida3	Resultado esperado
4	Descripción	Entrada4	Salida4	Resultado esperado

Figura 3.4: Registro pruebas de caja negra

3.2.2. Prueba de usabilidad

La prueba de usabilidad o SUS permite que el usuario evalúe la usabilidad de la aplicación mediante una encuesta, para la cual no se requiere conocimientos de software o de un lenguaje técnico para entender y responder las preguntas. Esta encuesta cuenta con 10 preguntas estándares, que serán respondidas en una escala del 1 al 5, tomando 1 como 'totalmente en desacuerdo', 2 como 'en desacuerdo', 3 como 'Neutro', 4 como 'de acuerdo' y 5 como 'totalmente de acuerdo'. A continuación se detallan las preguntas

1. Creo que me gustaría utilizar este sistema con frecuencia
2. Encontré el sistema innecesariamente complejo
3. Pensé que el sistema era fácil de usar
4. Creo que necesitaría el apoyo de un técnico para poder utilizar este sistema
5. Encontré que las diversas funciones de este sistema estaban bien integradas
6. Pensé que había demasiada inconsistencia en este sistema
7. Me imagino que la mayoría de la gente aprendería a utilizar este sistema muy rápidamente
8. Encontré el sistema muy complicado de usar
9. Me sentí muy seguro usando el sistema
10. Necesitaba aprender muchas cosas antes de empezar con este sistema

Para obtener el grado de aceptación de la aplicación se debe realizar el siguiente cálculo:

- Suma las respuestas de los enunciados impares y después resta 5, ejemplo: $(2 + 3 + 1 + 5 + 2) = 13 - 5 = 8$

- Suma las respuestas de los enunciados pares y resta ese total a 25, ejemplo: $(1 + 2 + 2 + 3 + 1) = 25 - 9 = 16$
- Suma ambos resultados y multiplíquelo por 2,5, ejemplo: $(8 + 16) * 2,5 = 60$

El puntaje final, oscila entre 0 y 100, permitiendo clasificar la usabilidad de la aplicación como Inaceptable (0-49), Mediocre (50-68) o Aceptable (69-100).

3.3. Resultados esperados

A continuación, se presentan los objetivos específicos con sus resultados esperados, además del medio para comprobarlo

1. OE1: Implementar una aplicación web que provea de un espacio de diseño en dos dimensiones (2D) orientado a muebles para que el fabricante pueda generar diseños personalizados en base a los requisitos del cliente.
 - Resultado esperado: Se espera un módulo de diseño funcional e interactivo, que le permita al mueblista arrastrar, soltar y redimensionar los módulos de muebles, logrando representar los requisitos del mueblista.
 - Pruebas para lograr resultado esperado: Pruebas de caja negra para la funcionalidad del Canvas.
2. OE2: Implementar el módulo de cubicación que apoye al fabricante en el cálculo de materiales y costos, agilizando el proceso de propuesta para el cliente.
 - Resultado esperado: Un algoritmo de cálculo integrado que procese automáticamente las dimensiones del diseño 2D y cuente con un listado de piezas y la cantidad de planchas de material necesarias.
 - Pruebas para lograr resultado esperado: Pruebas de caja negra comparando el resultado del algoritmo y un cálculo manual.
3. OE3: Desarrollar un repositorio de muebles que permita guardar, reutilizar y modificar diseños previamente realizados.
 - Resultado esperado: Un sistema de datos en MongoDB que permita crear, leer, actualizar y eliminar diseños de muebles.
 - Pruebas para lograr resultado esperado: Pruebas de caja negra para la integración de la base de datos.

4. OE4: Implementar una biblioteca de materiales, colores, grosores y accesorios que permita y facilite la personalización de los diseños.
 - Resultado esperado: Un catálogo digital gestionable en donde el mueblista pueda administrar los atributos de los materiales, los cuales se vinculan dinámicamente al cálculo de costos del diseño.
 - Pruebas para lograr resultado esperado: Pruebas de caja negra de ingreso y edición de datos.
5. OE5: Automatizar la generación y envío de cotizaciones detalladas e interactivas para que el cliente o mueblista elija entre un abanico de posibilidades.
 - Resultado esperado: La capacidad del sistema para generar un documento formal en formato PDF con los valores correctos de precio y material.
 - Pruebas para lograr resultado esperado: Pruebas de caja negra revisando los valores del PDF generado.
6. OE6: Reducir el tiempo en el proceso de fabricación de muebles a medida en un 15 % respecto al método manual.
 - Resultado esperado: Una disminución cuantificable de al menos un 15 % en el tiempo total requerido para completar el ciclo de Diseño + Cubicación + Cotización en comparación con el método tradicional manual.
 - Pruebas para lograr resultado esperado: Medición de tiempos comparando el flujo manual con el uso de la aplicación en un caso de prueba estándar.
7. OE7: Implementar un módulo de estadísticas que permita al mueblista visualizar datos relacionados a demanda e ingresos.
 - Resultado esperado: Un panel de visualización que presente datos claros sobre la cantidad de proyectos realizados y estimaciones de ingresos, filtrables por periodos de tiempo.
 - Pruebas para lograr resultado esperado: Pruebas de caja negra con los valores de los gráficos o datos visibles.
8. OE8: Evaluar la satisfacción y usabilidad de la aplicación a través de una encuesta de funcionalidad y usabilidad.

- Resultado esperado: Obtener una calificación de usabilidad superior a 68 puntos que es un nivel 'Aceptable' en la escala SUS, validando que la interfaz es intuitiva para el perfil del mueblista artesanal.
- Pruebas para lograr resultado esperado: Resultados de la encuesta SUS aplicada al usuario final tras las pruebas.

3.4. Resumen del capítulo

En este capítulo se detalló la forma en que se aplicará la metodología PXP en la propuesta. Además, se abordan las metodologías de evaluación y el funcionamiento de estas como pruebas de software para pruebas de caja negra y pruebas cualitativas para evaluar cuantitativamente con la encuesta SUS. Finalmente se abarcan los resultados esperados de los objetivos específicos.

En el siguiente capítulo se abarca la implementación de la metodología PXP, siguiendo sus etapas y explicando que se realizó en cada una.

4. Desarrollo

El presente capítulo está orientado a presentar la implementación de la metodología en desarrollo.

4.1. Análisis de factibilidad

Como se busca la viabilidad de la propuesta a continuación se presenta el análisis técnico y económico para revisar los recursos y gastos asociados al proyecto obteniendo que tan rentable es la propuesta.

4.1.1. Factibilidad técnica

El análisis de factibilidad técnica evalúa la disponibilidad de recursos de hardware y software necesarios para el desarrollo, implementación y operación del sistema propuesto. Dada la naturaleza de la solución y el perfil del usuario final, este análisis se desglosa en:

1. Requisitos del lado del cliente (Hardware y Software): A diferencia de software de diseño tradicional como AutoCAD, que exige estaciones de trabajo de alto rendimiento, la solución propuesta traslada la carga de procesamiento al navegador web mediante el uso de estándares HTML5.

Hardware: La aplicación requiere un navegador web moderno con soporte Canvas API (ej. Google Chrome, Mozilla Firefox, Safari o Edge en sus versiones actualizadas). Esto garantiza una alta disponibilidad, ya que estos navegadores son gratuitos y estándar en el mercado.

Recursos de Hardware: La librería seleccionada, Konva.js, realiza el renderizado de gráficos vectoriales en el cliente. Para garantizar una experiencia fluida, se estima un requisito mínimo de 4 GB de memoria RAM y un procesador de gama media o baja. Esto hace viable su uso en computadores portátiles estándar y teléfonos inteligentes no tan actuales, alineándose con la realidad económica de las PYMES objetivo.

2. Conectividad y Latencia en Entornos Rurales Considerando que el despliegue se orienta a zonas como Teno , donde la conexión a internet puede ser inestable o de ancho de banda limitado, la arquitectura de Single Page Application (SPA) implementada con React es técnicamente la más adecuada.

Justificación: Una vez cargado el bundle inicial de la aplicación, la interacción de diseño (arrastrar, soltar, redimensionar figuras) se ejecuta localmente en el navegador del usuario usando JavaScript, sin necesidad de realizar peticiones al servidor por cada movimiento. Esto minimiza la dependencia de una conexión de alta velocidad constante, requiriendo internet principalmente para el inicio de sesión y el guardado final del diseño, lo cual mitiga los riesgos de desconexión durante el proceso creativo.

3. Madurez y Disponibilidad del Stack Tecnológico El desarrollo se sustenta en el stack MERN (MongoDB, Express, React, Node.js), tecnologías de código abierto con licencias permisivas (MIT), lo que elimina barreras legales y de costos por licenciamiento.

Sostenibilidad: Estas tecnologías cuentan con documentación extensa y comunidades activas, asegurando la mantenibilidad del código a largo plazo.

Portabilidad: El uso de Docker para la contenerización permite que el sistema sea agnóstico a la infraestructura del servidor, garantizando que la aplicación pueda migrarse entre distintos proveedores de nube (Google Cloud, AWS, o servidores locales) sin conflictos de dependencias ni reescritura de código.

Conclusión Técnica: En base a lo expuesto, el proyecto es técnicamente factible. No requiere hardware especializado por parte del usuario final, se adapta a las condiciones de conectividad de zonas rurales gracias a su arquitectura SPA y utiliza tecnologías estándar de la industria que aseguran su viabilidad operativa y escalabilidad futura.

4.1.2. Factibilidad económica

La factibilidad económica evalúa los costos necesarios para la propuesta.

Cuadro 4.1: Factibilidad económica

Servicio	Tecnología	Costo
Frontend	React	\$0
Backend	Node.js	\$0
Base de datos	MongoDB	\$0
Servicio de correo	Email.js	\$0
Control de versiones	GitHub	\$0
Contenedor	Docker	\$0

Teniendo esta tabla de costos 4.1 se obtiene que el gasto de licencias de software es de \$0 pesos chilenos, ya que todas las herramientas tecnológicas que componen esta propuesta son gratuitas o de código abierto por lo que a nivel de gastos esta propuesta es muy viable ya que cuando empiece a escalar y se requiera de planes de pago a mayor escala el mismo proyecto ya lo podrá sustentar.

Teniendo en cuenta el despliegue de la aplicación en un servidor en la nube, se puede considerar el uso de servicios como Google Cloud Platform (GCP), Amazon Web Services (AWS) o Microsoft Azure. Estos servicios ofrecen planes gratuitos con ciertas limitaciones que podrían ser suficientes para un proyecto inicial o de pequeña escala.

Pero si se requiere de un plan pago, los costos pueden variar dependiendo del proveedor y los recursos utilizados. Por ejemplo, una instancia básica de propósito general con 2vCPU y 4 GB de RAM en un servidor de Google Cloud puede costar alrededor de \$57 USD, dependiendo de la configuración y el uso.

4.2. Diseño de software

Se presentan los elementos claves en el diseño de software, tales como arquitectura física, arquitectura lógica, modelo de datos y mockups principales.

4.2.1. Arquitectura física

La arquitectura física propuesta sigue el modelo cliente-servidor, el cual se ilustra en la figura 4.1 a continuación. En este modelo, el servidor actúa como el núcleo de procesamiento, recibiendo las solicitudes iniciadas por el cliente (frontend), ejecuta la lógica correspondiente en el backend y gestiona las consultas con la base de datos para finalmente retornar la respuesta requerida al usuario.

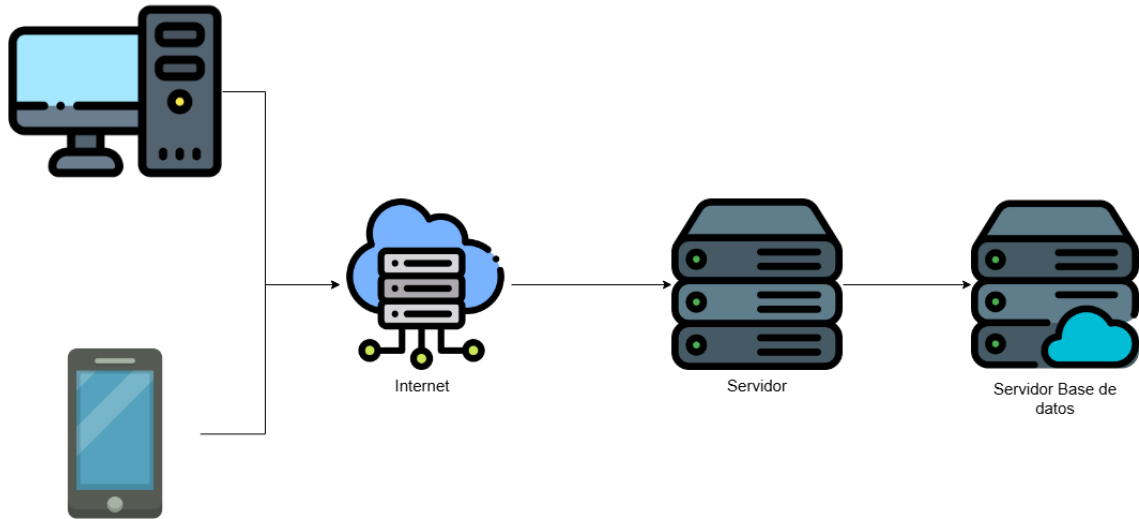


Figura 4.1: Arquitectura física

4.2.2. Arquitectura lógica

La arquitectura lógica propuesta sigue el patrón Modelo-Vista-Controlador (MVC). En la figura 4.2 se ilustra la comunicación bidireccional entre sus componentes. La capa 'Vistas' se encarga de mostrar la información al usuario a través de las interfaces, según los datos recibidos de la capa 'Controladores', la cual se encarga de manejar la lógica del sistema, procesando las peticiones POST, GET, PUT y DELETE que provienen de la vista. La capa 'Modelos' se usa para gestionar la estructura de datos y las reglas del sistema, permitiendo la gestión de información. Finalmente, la capa 'Base de datos' se encarga de guardar y entregar la información correctamente.

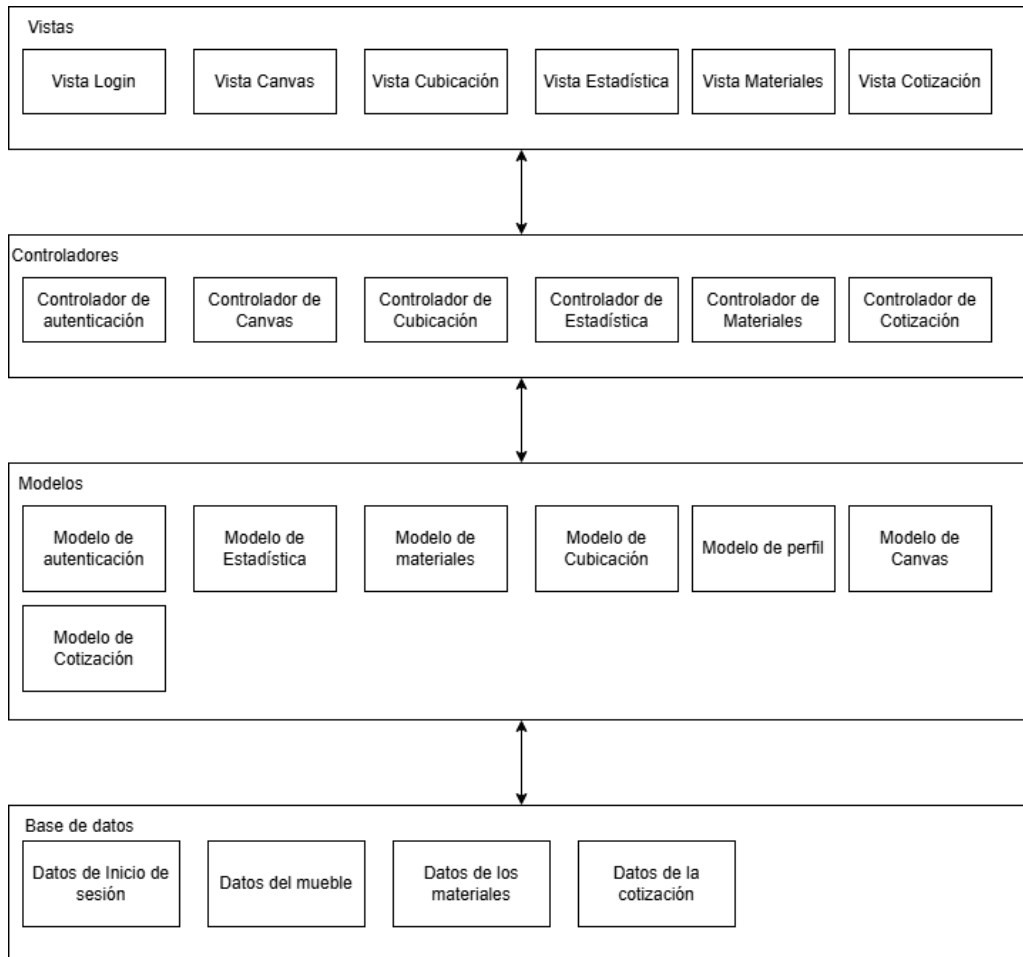


Figura 4.2: Arquitectura lógica

4.2.3. Modelo de datos

Como se mencionó anteriormente el esquema de base de datos a utilizar no es SQL por lo que hacer un diagrama ER no es la mejor representación, por lo que mediante la herramienta 'Hackolade', que es una herramienta de modelado de datos pude representar el funcionamiento de la base de datos NoSQL de una manera más visual y familiar entendiéndose como similar a un diagrama ER.

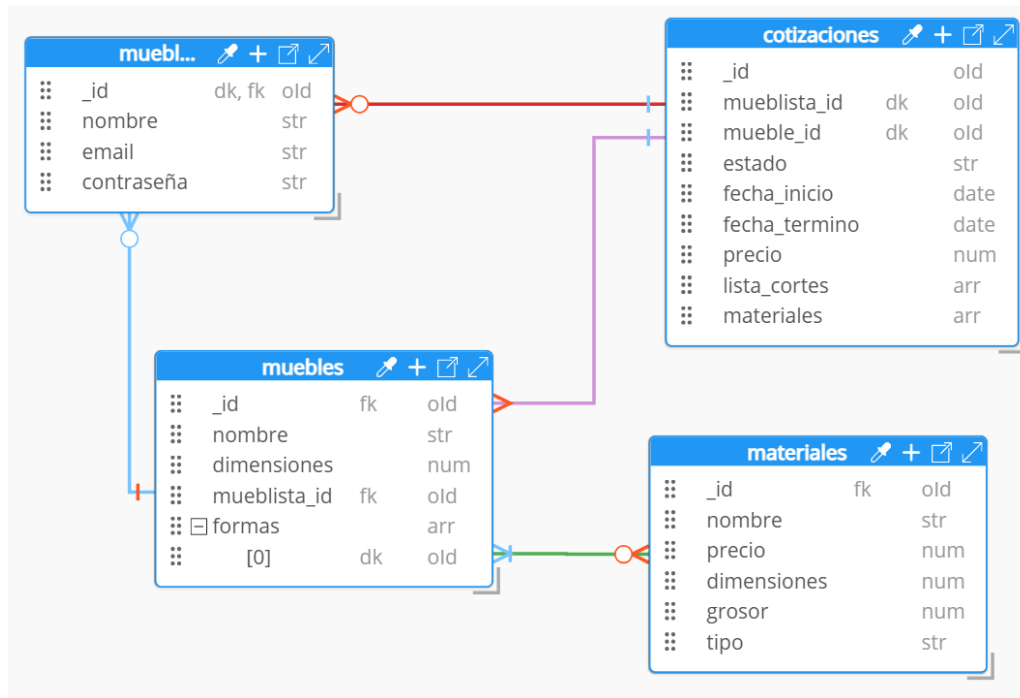


Figura 4.3: Modelo de datos

4.2.4. Diagrama de flujo

En la figura 4.4 se ilustra el flujo general del funcionamiento de la aplicación que es desde que inicia sesión, continuando con el diseño del mueble en donde lo modifica según los requisitos que requiera, para pasar a cubicación en donde se calcula la cantidad de material requerido y lo entrega en la vista para posteriormente pasar a la cotización en donde se hace el cálculo a partir de los precios en la base de datos.

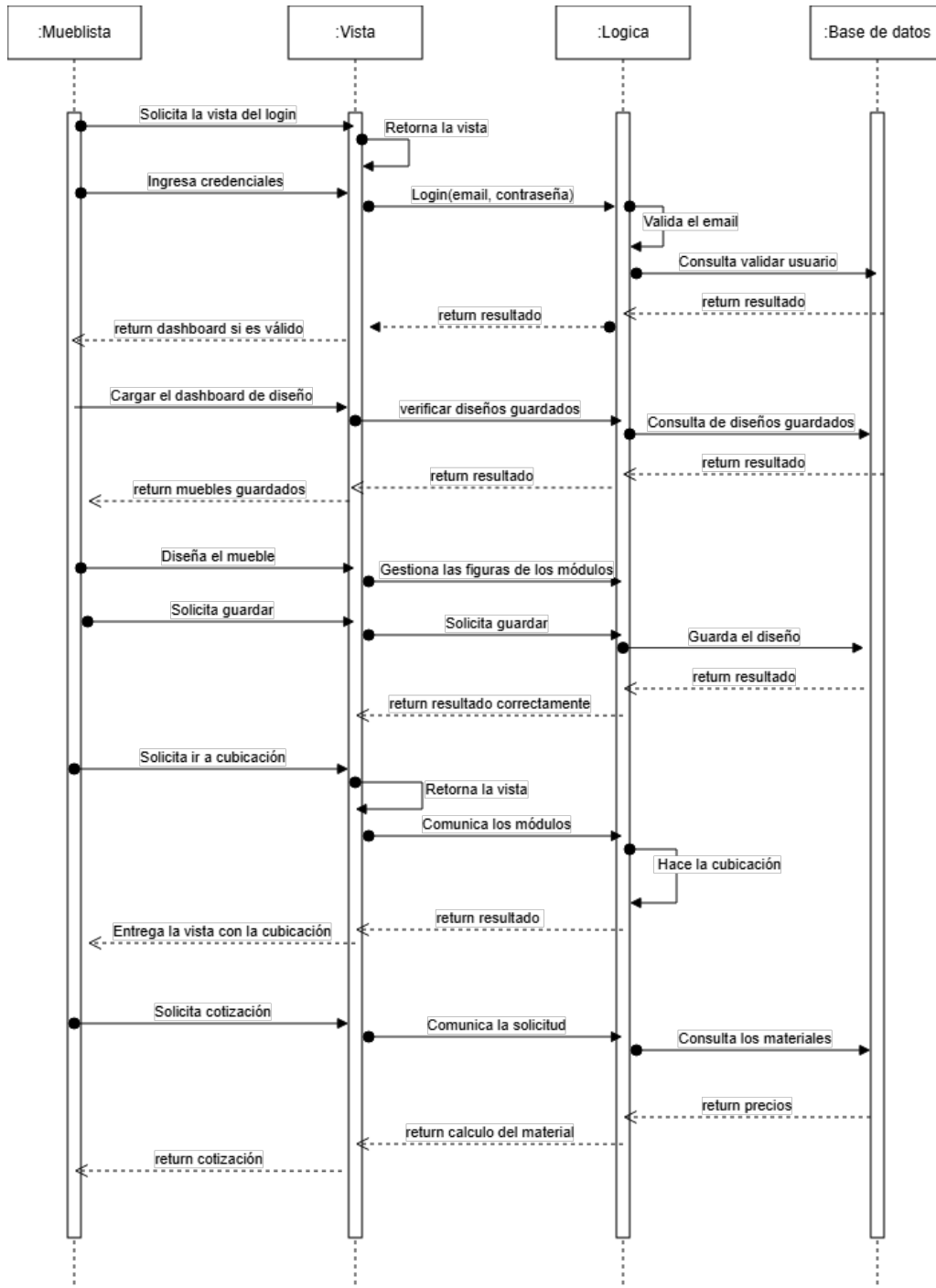


Figura 4.4: Diagrama de secuencia

4.2.5. Mockups

En este apartado se presentan algunos de los mockups propuestos durante el proceso de desarrollo del sistema, los cuales fueron diseñados y planificados en base a los requerimientos funcionales. Estos mockups tienen como función ilustrar las principales interfaces del sistema.

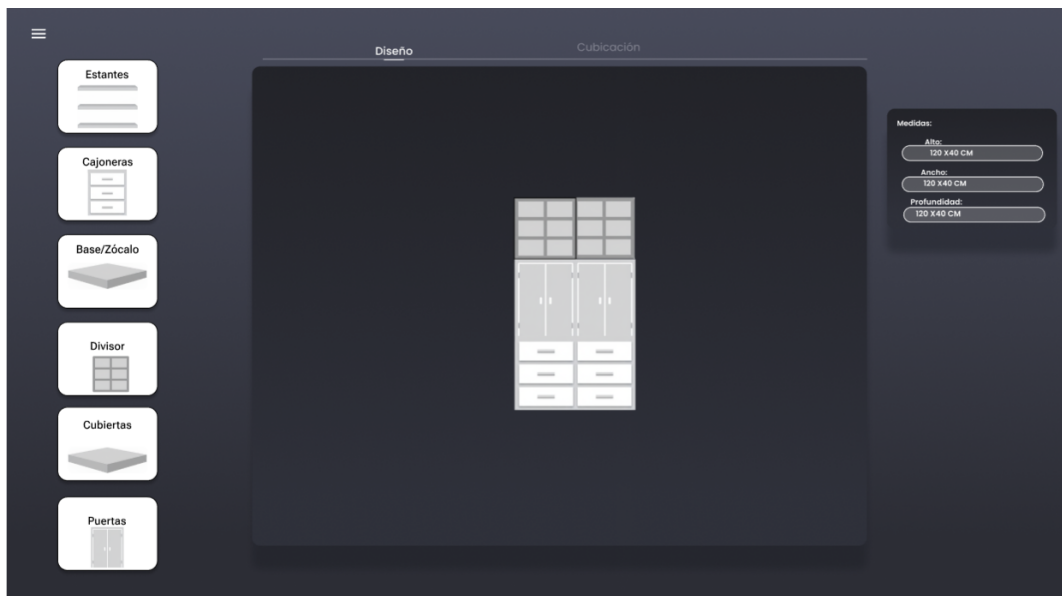


Figura 4.5: Mockup dashboard de diseño canvas en dispositivo de escritorio (PC, Notebook)

La figura 4.5 ilustra la vista luego de que un usuario se autentique, donde se presentan todos los componentes que puede interactuar dentro del sistema (siendo la vista pensada para computador).

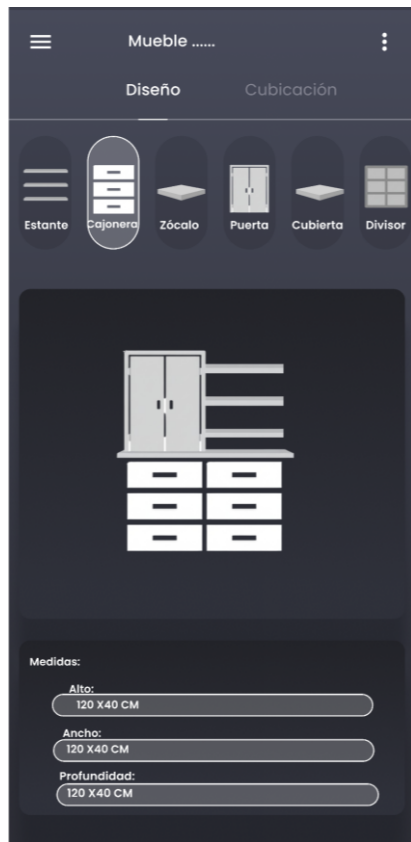


Figura 4.6: Mockup dashboard de diseño canvas en dispositivo móvil

La figura 4.6 ilustra la vista luego de que un usuario se autentique, donde se presentan todos los componentes que puede interactuar dentro del sistema (siendo la vista pensada para teléfono celular).

4.3. Iteraciones

A continuación, se detallan como se abordaron las iteraciones descritas en el Capítulo 3.1.3

4.3.1. Iteración 1

En esta iteración la historia de usuario a trabajar es HU01: Como usuario quiero iniciar sesión en el sistema. Para esto primero tuve que configurar el entorno del proyecto, es decir,

crear repositorios necesarios, descargar las tecnologías necesarias para comenzar el desarrollo. Una vez finalizada la instalación y configuración de las dependencias necesarias se da comienzo a la implementación de la metodología PXP.

Inicio de Iteración

En esta etapa, se crean y registran las tareas acordes a las historias de usuario a trabajar, en donde en el cuadro 4.1 se ilustra las tareas resultantes para la iteración.

Cuadro 4.2: Tareas asociadas a historias de usuario

HU	Tarea	Descripción
HU01	T01	Crear diagrama de secuencia
HU01	T02	Crear vista para el inicio de sesión
HU01	T03	Crear ruta para inicio de sesión
HU01	T04	Creación y testeo de pruebas unitarias

Diseño

En esta etapa, se diseña diagrama de secuencia de la funcionalidad de la iteración. A continuación, en la figura 4.6 se ilustra el flujo del inicio de sesión

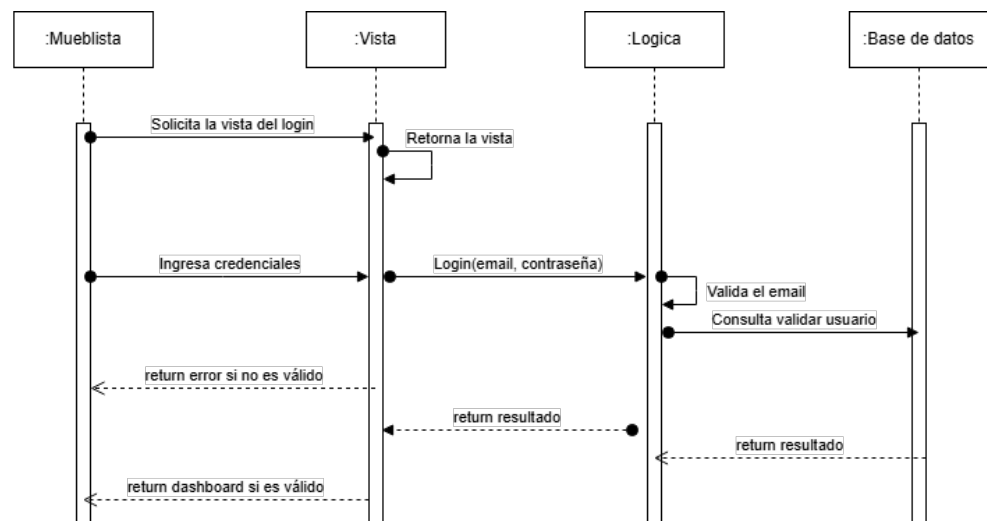


Figura 4.7: diagrama de secuencia HU01: login

Implementación

Para dar inicio a esta iteración se comienza instalando todas las dependencias y paquetes que en este caso es instalar React [?] con su adaptación a JavaScript, Konva.js [?], Mongoose para poder utilizar MongoDB [?] y Node.js [?] con express para poder hacer el backend. Una vez realizado eso procedemos a trabajar en HU01 que es el Login, para esto primero creamos la el formulario del login.

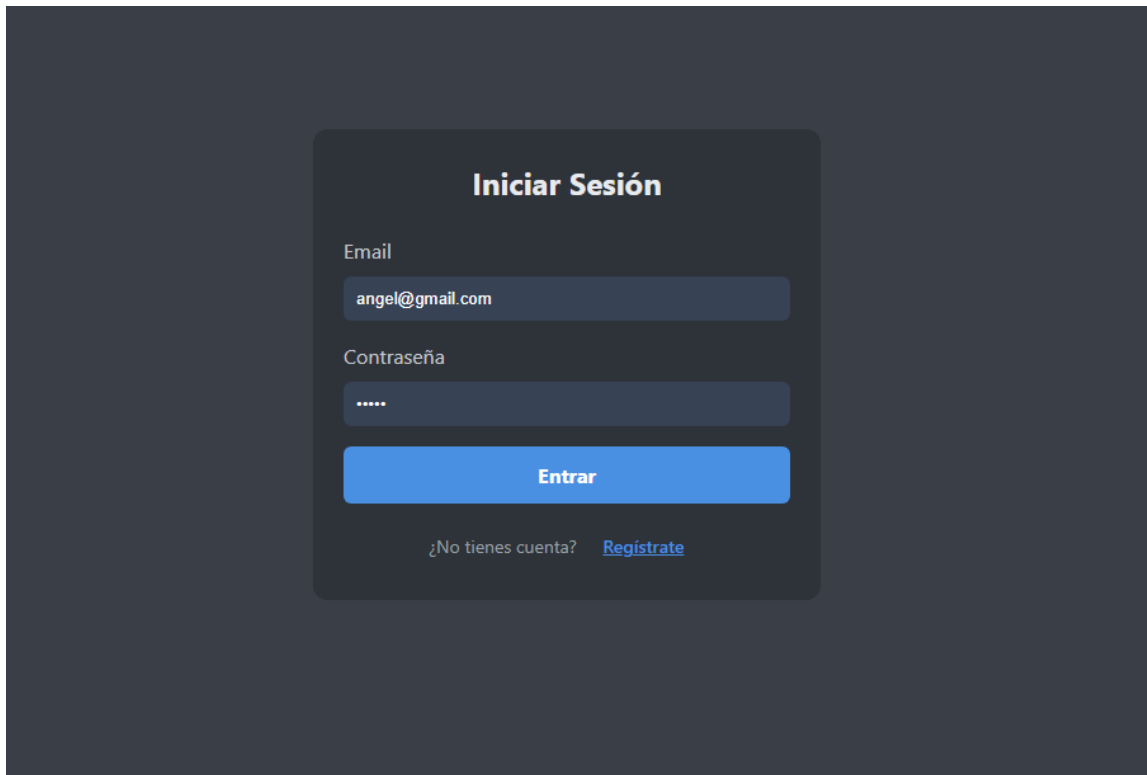


Figura 4.8: frontendlogin

También se tiene que crear el modelo con el cual se van a guardar los datos del mueblista para un próximo inicio de sesión.

```
backend > models > JS mueblista.js > ...
1  const mongoose = require('mongoose')
2
3  const muebistaSchema = new mongoose.Schema({
4    nombre: { type: String, required: true },
5    email: { type: String, required: true, unique: true },
6    contraseña: { type: String, required: true },
7  }, { timestamps: true })
8  |
9  module.exports = mongoose.model('Mueblista', muebistaSchema)
```

Figura 4.9: modelo de datos mueblista

Una vez creado el formulario y lo que va contener la base de datos se crea la lógica para poder utilizar el formulario, en donde se crea la función de AuthForm para autenticar el formulario y tenemos la variable para identificar si esta en el formulario de registro o de login/iniciar sesión, para después validar los datos y crear el token para mantener la sesión iniciada y poder guardar y cargar los diseños de muebles.

```
1  import React, { useState } from 'react'
2  import { login, register, saveToken } from '../services/auth.js'
3
4  export default function AuthForm({ onLogin }) {
5    const [isLogin, setIsLogin] = useState(true)
6    const [nombre, setNombre] = useState('')
7    const [email, setEmail] = useState('')
8    const [contrasena, setContrasena] = useState('')
9    const [error, setError] = useState('')
10
11    const handleSubmit = async (e) => {
12      e.preventDefault()
13      setError('')
14
15      try {
16        let result
17        if (isLogin) {
18          result = await login(email, contrasena)
19        } else {
20          result = await register(nombre, email, contrasena)
21        }
22
23        saveToken(result.token)
24        onLogin(result.user)
25      } catch (err) {
26        setError(err.message)
27      }
28    }
29  }
```

Figura 4.10: Backend login

Una vez creado el Frontend, el Backend y el modelo de la base de datos procedemos a crear las rutas para poder registrar y validar credenciales. Entonces, para poder hacer registro de un nuevo usuario se llama al método 'register' al cual se le entrega el nombre, email y contraseña (la cual se cifra con bcryptjs) para ser guardado en la base de datos y se le crea un token para mantener la sesión iniciada y validar que mueblista está usando el programa, todo esto se muestra en la figura 4.11. Para el login se crea la ruta consultando con los datos email y contraseña en donde son buscados en la base de datos para encontrar si corresponden y si son correctos se crea un token para poder mantener la sesión iniciada en el navegador como se muestra en la figura 4.12. Además, se crea la verificación del token para asegurar que usuario esta conectado al programa y esta interactuando con las

funcionalidades como se muestra en la figura 4.13 son solo validaciones en la cual se asegura la existencia del token y si no lo tiene se se asigna uno predeterminado.


```
backend > routes > JS authRoutes.js > ...
1  const express = require('express')
2  const bcrypt = require('bcryptjs')
3  const jwt = require('jsonwebtoken')
4  const Mueblista = require('../models/mueblista.js')
5
6  const router = express.Router()
7
8  // Registro
9  router.post('/register', async (req, res) => {
10   try {
11     const { nombre, email, contrasena } = req.body
12
13     const existingUser = await Mueblista.findOne({ email })
14     if (existingUser) {
15       return res.status(400).json({ error: 'El email ya está registrado' })
16     }
17
18     const hashedPassword = await bcrypt.hash(contrasena, 10)
19
20     const newUser = new Mueblista({
21       nombre,
22       email,
23       contrasena: hashedPassword,
24     })
25
26     await newUser.save()
27
28     const token = jwt.sign(
29       { id: newUser._id, email: newUser.email },
30       process.env.JWT_SECRET || 'secreto_super_seguro',
31       { expiresIn: '7d' }
32     )
33
34     res.status(201).json({
35       message: 'Usuario registrado correctamente',
36       token,
37       user: {
38         id: newUser._id,
39         nombre: newUser.nombre,
40         email: newUser.email,
41       },
42     })
43   } catch (err) {
44     res.status(500).json({ error: err.message })
45   }
46 })
```

Figura 4.11: Ruta login (registro)

```
46 router.post('/login', async (req, res) => {
47   try {
48     const { email, contrasena } = req.body
49
50     const user = await Mueblista.findOne({ email })
51     if (!user) {
52       return res.status(401).json({ error: 'Credenciales incorrectas' })
53     }
54
55     const isMatch = await bcrypt.compare(contrasena, user.contrasena)
56     if (!isMatch) {
57       return res.status(401).json({ error: 'Credenciales incorrectas' })
58     }
59
60     const token = jwt.sign(
61       { id: user._id, email: user.email },
62       process.env.JWT_SECRET || 'secreto_super_seguro',
63       { expiresIn: '7d' }
64     )
65
66     res.json({
67       message: 'Login exitoso',
68       token,
69       user: {
70         id: user._id,
71         nombre: user.nombre,
72         email: user.email,
73       },
74     })
75   } catch (err) {
76     res.status(500).json({ error: err.message })
77   }
78 })
79
```

Figura 4.12: Ruta login (validación)

```
91 router.get('/verify', async (req, res) => {
92   try {
93     const token = req.headers.authorization?.split(' ')[1]
94     if (!token) {
95       return res.status(401).json({ error: 'No token provided' })
96     }
97
98     const decoded = jwt.verify(token, process.env.JWT_SECRET || 'secreto_super_seguro')
99     const user = await Mueblista.findById(decoded.id).select('-contrasena')
100
101     if (!user) {
102       return res.status(404).json({ error: 'Usuario no encontrado' })
103     }
104
105     res.json({ user })
106   } catch (err) {
107     res.status(401).json({ error: 'Token inválido' })
108   }
109 })
```

Figura 4.13: Ruta login (verificación token)

Sistema de pruebas

Después de la implementación, se continua con las pruebas de la iteración en donde se evalúan los siguientes casos:

- Ingresar credenciales válidas.
- Ingresar credenciales inválidas.
- Ingresar una dirección de correo incompleta.

Retrospectiva

Al final esta iteración, se revisa el peso asignado a las historias de usuario, en esta iteración fue acertada y se completó la funcionalidad sin pendientes.

4.3.2. Iteración 2

En esta iteración la historia de usuario a trabajar es HU02: Como ususario quiero un canvas 2D para poder hacer diseños, HU03: Como usuario quiero crear, guardar, cargar y eliminar el diseño de un mueble, HU04: Como usuario quiero que el diseño se pueda hacer modular, HU05: Como usuario quiero que las medidas de los módulos agregados puedan

ser editables, HU06: Como usuario quiero que los módulos sean móviles para acomodarlos según la necesidad. Para esto da comienzo a la implementación de la metodología PXP.

Inicio de Iteración

En esta etapa, se crean y registran las tareas acordes a las historias de usuario a trabajar, en donde en el cuadro 4.2 se ilustra las tareas resultantes para la iteración.

Cuadro 4.3: Tareas asociadas a historias de usuario

HU	Tarea	Descripción
HU02, HU03, HU04, HU05, HU6	T01	Crear diagrama de secuencia
HU02	T02	Crear canvas 2D
HU02	T03	Crear ruta para el canvas
HU02	T04	Crear figuras para el canvas
HU03	T05	Crear botones de Crear, guardar, cargar y eliminar
HU03	T06	Crear rutas para los botones
HU04	T07	Modificar las propiedades de las figuras
HU05	T08	Creación de etiquetas para ingresar medidas
HU05	T09	Creación rutas para el ingreso de medidas
HU06	T010	Modificar las propiedades de las figuras
HU02, HU03, HU04, HU05, HU6	T011	Creación y testeo de pruebas unitarias

Diseño

En esta etapa, se diseña diagrama de secuencia de la funcionalidad de la iteración. A continuación, en la figura 4.7 se ilustra el flujo del inicio de sesión

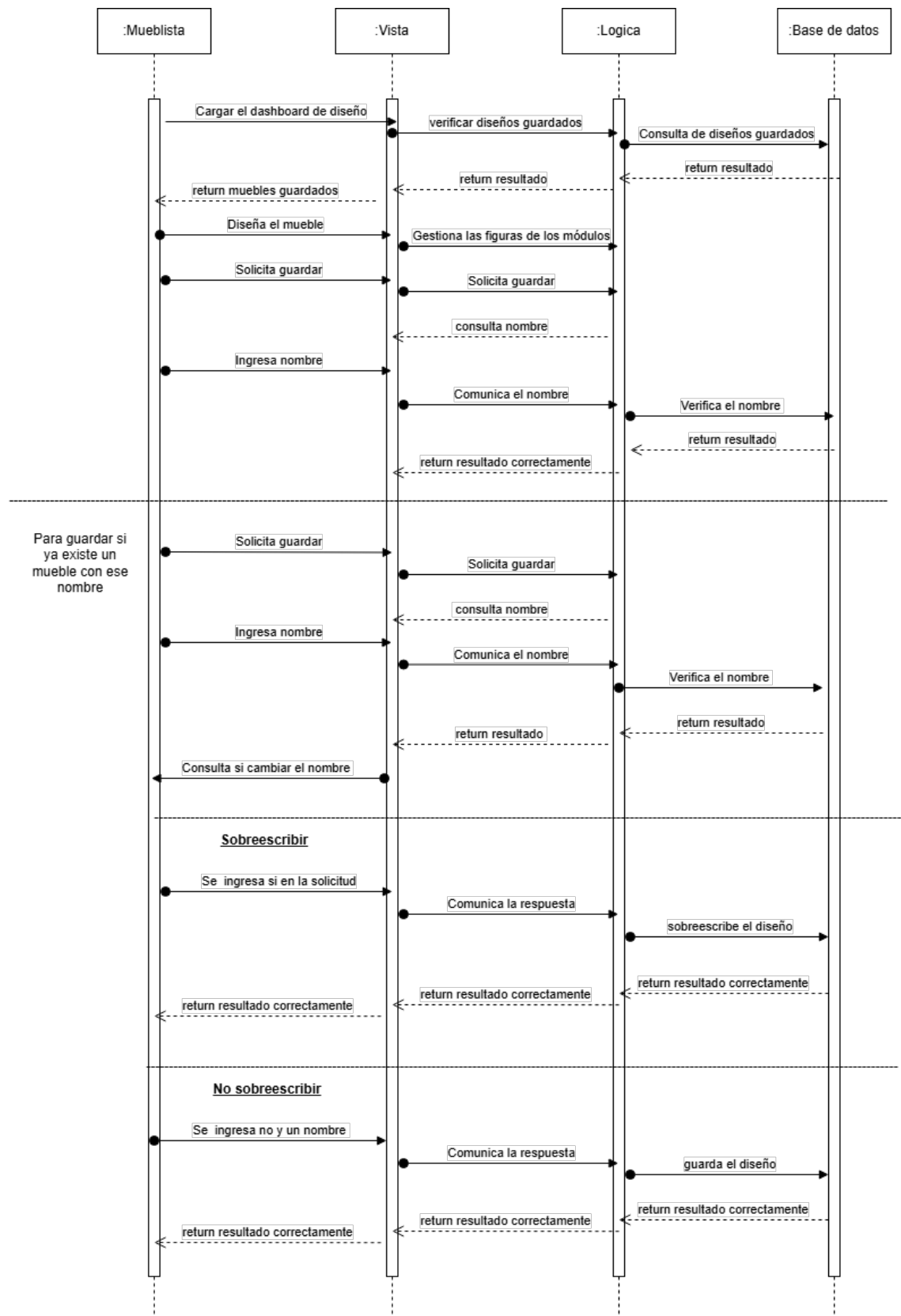


Figura 4.14: diagrama de secuencia HU02, HU03, HU04, HU05, HU6: Canvas

Implementación

En la figura 4.15 se muestra la función encargada de guardar los diseños de los muebles con sus correspondientes atributos en donde se guarda en un array las formas Creadas en el canvas.

```
const mongoose = require('mongoose')

const ShapeSchema = new mongoose.Schema({
  id: { type: Number, required: true },
  type: { type: String, required: true },
  x: { type: Number, required: true },
  y: { type: Number, required: true },
  width: { type: Number, required: true },
  height: { type: Number, required: true },
  depth: { type: Number, default: 20 },
  rotation: { type: Number, default: 0 },
  numCajones: { type: Number, default: null },
  numEstantes: { type: Number, default: null },
  numDivisores: { type: Number, default: null },
  numPuertas: { type: Number, default: null }
}, { _id: false })

const FurnitureSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  nombre: {
    type: String,
    required: true
  },
  shapes: [ShapeSchema]
}, {
  timestamps: true
})
```

Figura 4.15: Guardado de diseños

En la figura 4.16 se muestra las instancias de creación de cada forma para crear los módulos de los muebles en donde las medidas son en centímetros y cada figura cuenta con ancho, alto y profundidad (No se ilustra la profundidad ya que sirve para el futuro cálculo de material)

```
frontend > src > components > KonvaStage.jsx > checkCollision
1  import React, { useRef, useState, useEffect } from 'react'
2  import { Stage, Layer, Rect, Group, Text, Line } from 'react-konva'
3
4  const defaultSizes = {
5    estante: { width: 100, height: 1.5, depth: 70 },
6    cajonera: { width: 100, height: 70, depth: 70, numCajones: 3 },
7    modular: {
8      width: 100,
9      height: 70,
10     depth: 70,
11     numEstantes: 2,
12     numDivisores: 1,
13     numPuertas: 2
14   },
15   base: { width: 600, height: 10, depth: 65 },
16   divisor: { width: 1.5, height: 30, depth: 70 },
17   cubierta: { width: 110, height: 3, depth: 70 },
18   puerta: { width: 70, height: 70, depth: 1.5 },
19 }
```

Figura 4.16: Inicialización de figuras

En la figura 4.17 se muestra el funcionamiento del módulo de cajonera en donde pide el número de cajones para que el mueblista no tenga que hacer cada cajón, en donde se tiene por predefinido 3 cajones y si se quiere cambiar se hace la división de la altura entre la cantidad de cajones para dividir cómodamente el espacio.


```

{s.type === 'cajonera' && (() => {
  const numCajones = s.numCajones && s.numCajones > 0 ? s.numCajones : 3
  const drawerHeight = s.height / numCajones
  return (
    <>
      <Rect
        x={0}
        y={0}
        width={s.width}
        height={s.height}
        fill="■ #d9d9d9"
        stroke={selectedId === s.id ? '■ #4A90E2' : '■ #777'}
        strokeWidth={selectedId === s.id ? 3 : 2}
        shadowColor="black"
        shadowBlur={5}
        shadowOpacity={0.4}
        shadowOffsetY={3}
      />
      {Array.from({ length: numCajones }).map((_, i) => (
        <React.Fragment key={i}>
          <Line
            points={[0, (i + 1) * drawerHeight, s.width, (i + 1) * drawerHeight]}
            stroke="■ #555"
            strokeWidth={1}
          />
          <Rect
            x={s.width / 2 - 15}
            y={i * drawerHeight + drawerHeight / 2 - 3}
            width={30}
            height={6}
            fill="■ #444"
            cornerRadius={3}
          />
        </React.Fragment>
      ))}
    </>
  )
})()

```

Figura 4.17: Módulo de cajonera

En la figura 4.18 y 4.19 se muestra la lógica detrás del funcionamiento del módulo modular que es una figura que contiene el cubo de la figura, cantidad ajustable de estantes, divisores y puertas según se requiera, en donde funciona de la misma manera que en la cajonera se calcula el alto entre estantes se requieran, el ancho entre divisores y el ancho entre las puertas para así dividir el módulo y ser uno de las figuras más completas y

necesarias para hacer el diseño de un mueble.

```
{s.type === 'modular' && (() => {
  const numEstantes = s.numEstantes !== undefined && s.numEstantes !== null ? s.numEstantes : 0
  const numDivisores = s.numDivisores !== undefined && s.numDivisores !== null ? s.numDivisores : 0
  const numPuertas = s.numPuertas !== undefined && s.numPuertas !== null ? s.numPuertas : 0

  const estanteSpacing = numEstantes > 0 ? s.height / (numEstantes + 1) : 0
  const divisorSpacing = numDivisores > 0 ? s.width / (numDivisores + 1) : 0
  const puertaWidth = numPuertas > 0 ? s.width / numPuertas : 0

  return (
    <>
    <Rect
      x={0}
      y={0}
      width={s.width}
      height={s.height}
      fill="■ #f5f5f5"
      stroke={selectedId === s.id ? '■ #4A90E2' : '■ #888'}
      strokeWidth={selectedId === s.id ? 3 : 2}
      shadowColor="black"
      shadowBlur={5}
      shadowOpacity={0.3}
      shadowOffsetY={2}
    />

    {Array.from({ length: numEstantes }).map((_, i) => (
      <Line
        key={`estante-${i}`}
        points={[0, (i + 1) * estanteSpacing, s.width, (i + 1) * estanteSpacing]}
        stroke="■ #999"
        strokeWidth={3}
      />
    ))}

    {Array.from({ length: numDivisores }).map((_, i) => (
      <Line
        key={`divisor-${i}`}
        points={[i * divisorSpacing, 0, i * divisorSpacing, s.height]}
        stroke="■ #777"
        strokeWidth={3}
      />
    ))}
  )
}
```

Figura 4.18: Módulo principal

```

{Array.from({ length: numPuertas }).map((_, i) => {
  const manijaIzquierda = i % 2 === 1
  return (
    <React.Fragment key={`puerta-${i}`}>
      <Rect
        x={i * puertaWidth + 2}
        y={2}
        width={puertaWidth - 4}
        height={s.height - 4}
        fill="■ rgba(255,255,255,0.6)"
        stroke="■ #555"
        strokeWidth={2}
        cornerRadius={4}
      />

      <Rect
        x={manijaIzquierda
          ? i * puertaWidth + 8
          : i * puertaWidth + puertaWidth - 12
        }
        y={s.height / 2 - 15}
        width={4}
        height={30}
        fill="■ #333"
        cornerRadius={2}
      />
    </React.Fragment>
  )
})}
</>

```

Figura 4.19: Módulo principal puertas

En la figura 4.20 se muestra la función encargada de guardar los diseños de muebles en la cual se valida el 'id' del mueblista y el nombre del diseño, para recopilar los atributos

del diseño y ser guardados en la base de datos y ser accesible en una próxima instancia.

```
const express = require('express')
const router = express.Router()
const Furniture = require('../models/Furniture')
const authMiddleware = require('../middleware/auth')

// Crear nuevo diseño
router.post('/', authMiddleware, async (req, res) => {
  try {
    const { nombre, shapes } = req.body

    console.log('Guardando diseño:', { nombre, shapesCount: shapes.length })
    console.log('Shapes recibidos:', JSON.stringify(shapes, null, 2))
    console.log('User ID:', req.userId)

    if (!req.userId) {
      return res.status(401).json({ message: 'Usuario no autenticado' })
    }

    const furniture = new Furniture({
      userId: req.userId,
      nombre,
      shapes: shapes.map(shape => ({
        id: shape.id,
        type: shape.type,
        x: shape.x,
        y: shape.y,
        width: shape.width,
        height: shape.height,
        depth: shape.depth || 20,
        rotation: shape.rotation || 0,
        numCajones: shape.numCajones !== undefined ? shape.numCajones : null,
        numEstantes: shape.numEstantes !== undefined ? shape.numEstantes : null,
        numDivisores: shape.numDivisores !== undefined ? shape.numDivisores : null,
        numPuertas: shape.numPuertas !== undefined ? shape.numPuertas : null
      }))
    })

    await furniture.save()
    console.log('Diseño guardado exitosamente')

    res.status(201).json(furniture)
  } catch (error) {
    console.error('Error al guardar:', error)
    res.status(500).json({ message: 'Error al guardar el diseño', error: error.message })
  }
})
```

Figura 4.20: Ruta para guardar diseños de muebles

En la figura 4.21 se encarga de obtener todos los diseños de muebles que haya guardado el mueblista, esta función recopila según el 'id' y los carga para ser mostrados en un apartado del front.

```
router.get('/', authMiddleware, async (req, res) => {
  try {
    console.log('User ID:', req.userId)
    const designs = await Furniture.find({ userId: req.userId }).sort({ createdAt: -1 })
    console.log('Diseños cargados:', designs.length)
    res.json(designs)
  } catch (error) {
    console.error('Error al cargar diseños:', error)
    res.status(500).json({ message: 'Error al cargar diseños', error: error.message })
  }
})
```

Figura 4.21: Ruta para cargar diseños de muebles

En la figura 4.22 se encarga eliminar el diseño de mueble seleccionado mediante la solicitud delete, a lo cual se valida la existencia del diseño para poder ser eliminado.

```
router.delete('/:id', authMiddleware, async (req, res) => {
  try {
    console.log('User ID:', req.userId)
    const furniture = await Furniture.findOneAndDelete({
      _id: req.params.id,
      userId: req.userId
    })
  }

  if (!furniture) {
    return res.status(404).json({ message: 'Diseño no encontrado' })
  }

  console.log('Diseño eliminado:', req.params.id)
  res.json({ message: 'Diseño eliminado correctamente' })
} catch (error) {
  console.error('Error al eliminar:', error)
  res.status(500).json({ message: 'Error al eliminar el diseño', error: error.message })
}
})
```

Figura 4.22: Ruta para eliminar diseños de muebles

En la figura 4.23 se muestra la función encargada de la actualización de diseños en donde se obtienen cuando se quiere sobrescribir desde el Frontend. Se basa en recopilar

todos los datos del diseño y buscar el diseño con el mismo nombre para actualizarlo o sobrescribirlo.

```
router.put('/:id', authMiddleware, async (req, res) => {
  try {
    const { nombre, shapes } = req.body

    console.log('Actualizando diseño:', req.params.id)
    console.log('Shapes recibidos:', JSON.stringify(shapes, null, 2))
    console.log('User ID:', req.userId)

    const furniture = await Furniture.findOneAndUpdate(
      { _id: req.params.id, userId: req.userId },
      {
        nombre,
        shapes: shapes.map(shape => ({
          id: shape.id,
          type: shape.type,
          x: shape.x,
          y: shape.y,
          width: shape.width,
          height: shape.height,
          depth: shape.depth || 70,
          rotation: shape.rotation || 0,
          numCajones: shape.numCajones !== undefined ? shape.numCajones : null,
          numEstantes: shape.numEstantes !== undefined ? shape.numEstantes : null,
          numDivisores: shape.numDivisores !== undefined ? shape.numDivisores : null,
          numPuertas: shape.numPuertas !== undefined ? shape.numPuertas : null
        })))
      },
      { new: true }
    )

    if (!furniture) {
      return res.status(404).json({ message: 'Diseño no encontrado' })
    }

    console.log('Diseño actualizado exitosamente')
    res.json(furniture)
  } catch (error) {
    console.error('Error al actualizar:', error)
    res.status(500).json({ message: 'Error al actualizar el diseño', error: error.message })
  }
})
```

Figura 4.23: Ruta para actualizar diseños de muebles

Sistema de pruebas

Después de la implementación, se continua con las pruebas de la iteración en donde se evalúan los siguientes casos:

- Crear figuras.
- Mover figuras.
- Editar medidas de las figuras.
- Guardar diseños de muebles.
- Cargar diseños de muebles.
- Eliminar diseños de figuras.
- Eliminar diseños de muebles.
- Guardar diseños de muebles con el mismo nombre.

Retrospectiva

Al final esta iteración, se revisa el peso asignado a las historias de usuario, en esta iteración fue subestimado y no se ha terminado completamente la iteración.

4.3.3. Iteración 3

En esta iteración las historias de usuario a trabajar son HU07: Como usuario quiero que el sistema me calcule la cantidad de material, y HU08: Como usuario quiero que se muestre un listado de las piezas que se van a utilizar. Esta iteración corresponde al módulo de cubicación, que es uno de los más importantes del sistema ya que automatiza el cálculo de materiales a partir del diseño realizado en el canvas.

Inicio de Iteración

En esta etapa, se crean y registran las tareas acordes a las historias de usuario a trabajar, en donde en el cuadro 4.3 se ilustran las tareas resultantes para la iteración.

Cuadro 4.4: Tareas de la iteración 3

HU	Tarea	Descripción
HU07, HU08	T01	Crear diagrama de secuencia del módulo de cubicación
HU07	T02	Implementar algoritmo de generación de piezas por módulo
HU07	T03	Implementar algoritmo de cálculo de planchas necesarias
HU08	T04	Crear vista del panel de cubicación con listado de piezas
HU07, HU08	T05	Creación y testeo de pruebas unitarias

Diseño

En esta etapa se diseña el diagrama de secuencia de las funcionalidades de la iteración. El flujo principal comienza cuando el mueblista solicita calcular el material desde el canvas, el frontend recorre todas las figuras del diseño, genera la lista de piezas por módulo y calcula cuántas planchas de melamina se requieren para cubrir el área total de cortes.

Implementación

Para la implementación de este módulo se desarrolló el archivo `cubicacion.js` en el frontend, el cual contiene toda la lógica de cálculo. Lo primero que se realizó fue implementar la función `generateStructuredCuts`, que recorre cada figura del canvas y según su tipo de módulo (cajonera, modular, estante, etc.) genera las piezas correspondientes con sus dimensiones de ancho, alto y cantidad.

En el caso de la cajonera, la función calcula automáticamente las piezas de Techo/Piso, Laterales, Frente (uno por cajón) y Fondo a partir de las dimensiones del módulo y la cantidad de cajones configurada. Para el módulo modular, que es el más complejo, calcula adicionalmente las piezas de estantes, divisores y puertas según los parámetros de configuración. Las puertas descuentan 3 mm del ancho y del alto para dejar holgura de montaje.

Una vez generadas todas las piezas, se ejecuta el algoritmo de cálculo de planchas `boardsNeeded`, el cual estima la cantidad de planchas de melamina de 250×183 cm nece-

sarias para fabricar el mueble usando el área total de las piezas. El resultado se despliega en el componente `CubicacionPanel`, que lista las piezas agrupadas por módulo con su cantidad y dimensiones, y muestra al final el total de planchas requeridas.

Sistema de pruebas

Después de la implementación, se continúa con las pruebas de la iteración en donde se evalúan los siguientes casos:

- Cajonera genera exactamente 4 tipos de piezas (Techo/Piso, Frente, Laterales y Fondo).
- La pieza Frente de una cajonera tiene cantidad igual al número de cajones configurado.
- Las dimensiones de los Laterales corresponden a la profundidad y al alto del mueble.
- El Techo/Piso utiliza el ancho y la profundidad del mueble.
- Módulo sin estantes ni puertas genera 3 tipos de piezas.
- Módulo con estantes genera la pieza Estantes con la cantidad correcta.
- Las puertas descuentan correctamente 3 mm del ancho y del alto.
- El área total de las piezas no supera el área teórica de las planchas calculadas.
- Una sola pieza pequeña requiere exactamente 1 plancha.

Retrospectiva

Al final de esta iteración se revisó el peso asignado y la estimación fue ajustada, ya que el algoritmo de generación de piezas requirió más casos de borde de los previstos inicialmente. La iteración se completó con todas las funcionalidades operativas.

4.3.4. Iteración 4

En esta iteración las historias de usuario a trabajar son HU09: Como usuario quiero que el sistema pueda generar cotizaciones a partir del diseño, HU10: Como usuario quiero que se vea una previsualización de la cotización, y HU12: Como usuario quiero que las

cotizaciones se puedan exportar como PDF. Esta iteración corresponde al módulo de gestión de cotizaciones, que permite al mueblista generar, visualizar y exportar una cotización formal a partir de un diseño con su cubicación ya calculada.

Inicio de Iteración

En esta etapa, se crean y registran las tareas acordes a las historias de usuario a trabajar, en donde en el cuadro 4.4 se ilustran las tareas resultantes para la iteración.

Cuadro 4.5: Tareas de la iteración 4

HU	Tarea	Descripción
HU09, HU10, HU12	T01	Crear diagrama de secuencia del módulo de cotizaciones
HU09	T02	Crear modelo de datos de cotización en MongoDB
HU09	T03	Crear rutas para crear, listar, actualizar estado y eliminar cotizaciones
HU10	T04	Crear vista de previsualización de cotización
HU12	T05	Implementar exportación a PDF con jsPDF
HU09, HU10, HU12	T06	Creación y testeo de pruebas unitarias

Diseño

El flujo del módulo de cotizaciones comienza cuando el mueblista ha completado el diseño y la cubicación. A partir de los datos calculados en el frontend (lista de cortes, resumen de materiales y precio total), se envía una petición POST al backend para crear y persistir la cotización en la base de datos. Luego el sistema permite previsualizarla y exportarla como PDF.

Implementación

Para el backend se creó el modelo de datos `cotizacion.js` en MongoDB, el cual almacena el identificador del mueblista, la referencia al diseño de mueble, el nombre y correo del cliente, el precio total, la lista de cortes y el resumen de materiales usados. También se registra el estado de la cotización, que puede ser Pendiente, En Proceso o Completado.

Se implementó el archivo `cotizacionRoutes.js` con las rutas correspondientes: GET para listar las cotizaciones del usuario autenticado ordenadas por fecha, GET por ID para obtener una cotización específica, POST para crear una nueva cotización validando que el diseño de mueble exista y pertenezca al usuario, PATCH para actualizar el estado de la cotización y DELETE para eliminarla. Todas las rutas están protegidas por el middleware de autenticación JWT.

Para la exportación a PDF se implementó el servicio `pdfCotizacion.js` en el frontend usando la librería jsPDF. El documento generado incluye encabezado con logo, datos del mueblista y del cliente, tabla con el listado de cortes, resumen de materiales con sus costos y el precio total de la cotización. El estado de la cotización se representa visualmente con un color distintivo según sea Pendiente, En Proceso o Completado.

Sistema de pruebas

Después de la implementación, se continúa con las pruebas de la iteración en donde se evalúan los siguientes casos:

- Crear una cotización con datos válidos y verificar que se guarda correctamente.
- Intentar crear una cotización con un diseño que no pertenece al usuario.
- Listar las cotizaciones del usuario autenticado.
- Cambiar el estado de una cotización de Pendiente a En Proceso.
- Cambiar el estado a Completado y verificar que se registra la fecha de término.
- Intentar cambiar a un estado inválido.
- Exportar la cotización a PDF y verificar que los valores son correctos.
- Eliminar una cotización y verificar que ya no aparece en el listado.

Retrospectiva

Al final de esta iteración se revisó el peso asignado y el esfuerzo fue el esperado. La iteración se completó sin pendientes, con todas las funcionalidades del módulo de cotizaciones operativas.

4.3.5. Iteración 5

En esta iteración las historias de usuario a trabajar son HU14: Como usuario quiero disponer de una biblioteca de materiales y HU15: Como usuario quiero poder agregar, editar y eliminar materiales de la biblioteca. Esta iteración corresponde al módulo de gestión de la biblioteca de materiales, que permite al mueblista administrar los materiales con sus precios y atributos para que sean utilizados en el cálculo de cotizaciones.

Inicio de Iteración

En esta etapa, se crean y registran las tareas acordes a las historias de usuario a trabajar, en donde en el cuadro 4.5 se ilustran las tareas resultantes para la iteración.

Cuadro 4.6: Tareas de la iteración 5

HU	Tarea	Descripción
HU14, HU15	T01	Crear diagrama de secuencia del módulo de materiales
HU14	T02	Crear modelo de datos de material en MongoDB
HU15	T03	Crear rutas para crear, listar, actualizar y eliminar materiales
HU14	T04	Crear vista de la biblioteca de materiales
HU14, HU15	T05	Creación y testeo de pruebas unitarias

Diseño

El módulo de biblioteca de materiales permite al mueblista mantener un catálogo digital propio. Cada material tiene un nombre, categoría, precio y atributos específicos según la categoría. Las categorías disponibles son: **material** (planchas como melamina, con tipo y

dimensiones), **accesorio** (herrajes, con tipo de accesorio), **tapa-canto** (cantos de PVC o aluminio) y **cubierta**. Estos materiales quedan vinculados al cálculo de costos en el módulo de cotizaciones.

Implementación

Se creó el modelo de datos `material.js` en MongoDB, que almacena para cada material el identificador del usuario, el nombre, la categoría, el precio, la unidad de medida, el grosor, el tipo y los atributos específicos de cada categoría.

En el backend se implementó el archivo `materialRoutes.js` con las rutas para listar, crear, actualizar y eliminar materiales, todas protegidas con el middleware de autenticación. El listado soporta filtros por categoría y tipo, lo que permite al frontend cargar solo los materiales necesarios para cada contexto. Se implementó también la función de validación `validateMaterialPayload` que verifica que los campos obligatorios estén presentes según la categoría del material, por ejemplo que un material de tipo 'accesorio' tenga el campo `accesorio_tipo`.

En el frontend se creó el componente `MaterialLibrary.jsx`, que muestra los materiales agrupados por categoría, con opciones para agregar nuevos materiales mediante un formulario, editar los existentes y eliminarlos con un diálogo de confirmación.

Sistema de pruebas

Después de la implementación, se continúa con las pruebas de la iteración en donde se evalúan los siguientes casos:

- Agregar un material con todos los campos válidos.
- Intentar agregar un material sin nombre.
- Intentar agregar un material con una categoría inválida.
- Agregar un accesorio sin el campo tipo de accesorio.
- Intentar ingresar un precio negativo.
- Editar el precio de un material existente y verificar que se actualiza.
- Eliminar un material confirmando la acción.

- Cancelar la eliminación de un material.
- Intentar eliminar un material de otro usuario mediante la API.

Retrospectiva

Al final de esta iteración se revisó el peso asignado y el esfuerzo fue el esperado. La iteración se completó sin pendientes.

4.3.6. Iteración 6

En esta iteración las historias de usuario a trabajar son HU16: Como usuario quiero ver estadísticas básicas y HU17: Como usuario quiero que se puedan filtrar las estadísticas por periodos. Esta iteración corresponde al módulo de estadísticas, que permite al mueblista visualizar indicadores clave y gráficos sobre el historial de diseños y cotizaciones.

Inicio de Iteración

En esta etapa, se crean y registran las tareas acordes a las historias de usuario a trabajar, en donde en el cuadro 4.6 se ilustran las tareas resultantes para la iteración.

Cuadro 4.7: Tareas de la iteración 6

HU	Tarea	Descripción
HU16, HU17	T01	Crear diagrama de secuencia del módulo de estadísticas
HU16	T02	Implementar los KPIs del panel de estadísticas
HU16	T03	Implementar los gráficos del panel de estadísticas
HU17	T04	Implementar el filtro por año
HU16, HU17	T05	Creación y testeo de pruebas de caja negra

Diseño

El módulo de estadísticas obtiene los datos directamente de las cotizaciones y diseños ya guardados del usuario. Al cargar el panel, se hace una petición al endpoint de cotizaciones y se procesan los datos en el cliente para construir los indicadores y gráficos. Se implementó

un selector de año para filtrar los datos por periodo, actualizando todos los indicadores y gráficos en tiempo real al cambiar el año seleccionado.

Implementación

Se implementó el componente `StatsPanel.jsx` en el frontend, que es el encargado de centralizar toda la lógica y visualización de estadísticas. Al cargar el componente se obtienen las cotizaciones del usuario mediante el servicio `listCotizaciones` y los diseños mediante el servicio de muebles. A partir de estos datos se calculan los siguientes KPIs: total de diseños guardados, total de cotizaciones, monto acumulado de cotizaciones, precio promedio y total de módulos usados.

Para la visualización se implementaron gráficos de barras horizontales para mostrar los diseños creados por mes, la distribución de cotizaciones por estado (Pendiente, En Proceso, Completado) y los tipos de módulos más utilizados en los diseños. Cuando el usuario no tiene datos registrados, los KPIs muestran el valor 0 y los gráficos muestran un mensaje de 'Sin datos'. El filtro de año permite al mueblista comparar su desempeño entre distintos periodos.

Sistema de pruebas

Después de la implementación, se continúa con las pruebas de la iteración en donde se evalúan los siguientes casos:

- El panel de estadísticas es accesible para un usuario con sesión iniciada.
- El KPI de diseños totales muestra el conteo correcto.
- El KPI de cotizaciones totales muestra el conteo correcto.
- El KPI de monto acumulado suma correctamente los valores.
- El KPI de precio promedio calcula correctamente el promedio.
- El gráfico de diseños por mes muestra barras para los meses correspondientes.
- El gráfico de cotizaciones por estado refleja la distribución correcta.
- Acceder al panel sin datos muestra todos los KPIs en 0 y mensajes de sin datos.
- Filtrar por año actual muestra solo los datos del año actual.

- Filtrar por año anterior muestra solo los datos del año anterior.
- Seleccionar un periodo sin datos muestra los KPIs en 0.

Retrospectiva

Al final de esta iteración se revisó el peso asignado y el esfuerzo fue el esperado. El módulo de estadísticas quedó operativo con todas sus funcionalidades.

4.3.7. Iteración 7

En esta iteración la historia de usuario a trabajar es HU13: Como usuario quiero que el sistema permita enviar cotizaciones por correo. Esta iteración corresponde a la implementación del envío de cotizaciones por correo electrónico al cliente, integrando el servicio Email.js directamente desde el frontend.

Inicio de Iteración

En esta etapa, se crean y registran las tareas acordes a las historias de usuario a trabajar, en donde en el cuadro 4.7 se ilustran las tareas resultantes para la iteración.

Cuadro 4.8: Tareas de la iteración 7

HU	Tarea	Descripción
HU13	T01	Configurar plantilla de correo en Email.js
HU13	T02	Implementar lógica de envío desde el frontend
HU13	T03	Integrar el envío de correo al flujo de cotizaciones
HU13	T04	Testeo del envío de correo

Diseño

El flujo de envío de correo comienza cuando el mueblista ingresa el correo del cliente en la vista de cotización y presiona el botón de enviar. El frontend llama al servicio Email.js con los datos de la cotización usando una plantilla predefinida en la plataforma de Email.js, la cual incluye el resumen de la cotización y el precio total. No se requiere configurar ni mantener un servidor de correo propio ya que Email.js actúa como intermediario.

Implementación

Para la implementación de esta funcionalidad se utilizó la librería `@emailjs/browser` en el frontend. Se creó una plantilla de correo en la plataforma de Email.js que incluye los datos del cliente, el detalle del mueble cotizado, el listado de materiales y el precio total. Desde la vista de cotizaciones, cuando el mueblista ingresa el correo del cliente y confirma el envío, se llama a la función `emailjs.send` con los parámetros correspondientes de la plantilla. En caso de éxito se notifica al usuario con un mensaje de confirmación, y en caso de error se muestra el mensaje correspondiente.

Se eligió Email.js ya que su plan gratuito incluye 200 correos mensuales con 2 plantillas, lo cual es suficiente para las dimensiones del proyecto y no requiere configuración de servidor.

Sistema de pruebas

Después de la implementación, se continúa con las pruebas de la iteración en donde se evalúan los siguientes casos:

- Enviar una cotización a un correo válido y verificar la recepción.
- Intentar enviar sin ingresar un correo de destino.
- Verificar que el correo recibido contiene los datos correctos de la cotización.
- Verificar que se muestra notificación de éxito tras el envío.

Retrospectiva

Al final de esta iteración se revisó el peso asignado y el esfuerzo fue menor al estimado, ya que Email.js simplificó bastante la integración. La iteración se completó sin pendientes.

4.3.8. Iteración 8

En esta iteración la historia de usuario a trabajar es HU03: Como usuario quiero crear, guardar, cargar y eliminar el diseño de un mueble. Si bien las rutas del backend para guardar y cargar muebles fueron implementadas en la Iteración 2, en esta iteración se desarrolló la vista dedicada de biblioteca de muebles, que permite al mueblista acceder de forma centralizada a todos sus diseños guardados, cargarlos en el canvas o eliminarlos.

Inicio de Iteración

En esta etapa, se crean y registran las tareas acordes a las historias de usuario a trabajar, en donde en el cuadro 4.8 se ilustran las tareas resultantes para la iteración.

Cuadro 4.9: Tareas de la iteración 8

HU	Tarea	Descripción
HU03	T01	Diseñar la vista de biblioteca de muebles
HU03	T02	Implementar la vista con listado de diseños guardados
HU03	T03	Implementar la carga de un diseño desde la biblioteca al canvas
HU03	T04	Implementar la eliminación de diseños desde la biblioteca
HU03	T05	Creación y testeo de pruebas de caja negra

Diseño

La biblioteca de muebles es una vista que lista todos los diseños guardados por el mueblista, mostrando el nombre y la fecha de creación de cada uno. Desde esta vista el mueblista puede seleccionar un diseño para cargarlo en el canvas y continuar editándolo, o eliminarlo con un diálogo de confirmación. La integración con el backend reutiliza las rutas de diseños ya implementadas en la Iteración 2 (GET para listar, DELETE para eliminar).

Implementación

Para esta iteración se integró la vista de biblioteca de muebles en la interfaz principal de la aplicación. Al acceder a esta sección, se hace una petición GET al endpoint de diseños del usuario, obteniendo el listado completo ordenado por fecha de creación. Cada diseño se muestra como una tarjeta con el nombre del mueble y la fecha en que fue guardado.

Al presionar el botón de cargar, las figuras del diseño seleccionado se deserializan y se cargan en el canvas de Konva, reemplazando el diseño actual. Para la eliminación se muestra un diálogo de confirmación antes de ejecutar la petición DELETE al backend, para evitar eliminaciones accidentales. Una vez eliminado el diseño, la biblioteca se actualiza

automáticamente.

Sistema de pruebas

Después de la implementación, se continúa con las pruebas de la iteración en donde se evalúan los siguientes casos:

- Cargar la biblioteca con diseños guardados y verificar que se listan correctamente.
- Seleccionar un diseño y cargarlo en el canvas.
- Verificar que el canvas muestra correctamente el diseño cargado.
- Eliminar un diseño confirmando la acción.
- Cancelar la eliminación de un diseño.
- Verificar que la biblioteca se actualiza tras una eliminación.
- Acceder a la biblioteca con el usuario sin diseños guardados.

Retrospectiva

Al final de esta iteración se revisó el peso asignado y el esfuerzo fue el esperado, ya que la mayor parte de la lógica de backend ya estaba implementada. La iteración se completó sin pendientes, cerrando así el ciclo de desarrollo de los módulos principales de la aplicación.

5. Conclusiones y trabajo futuro

En este capítulo se abarcan las conclusiones obtenidas del trabajo realizado y de lo que queda por realizar en el proyecto.

5.1. Cumplimiento de Objetivos

Al término del desarrollo se completaron las 8 iteraciones planificadas, implementando la totalidad de los módulos del sistema. A continuación se detalla el estado de cumplimiento de cada objetivo específico:

- **OE1 — Canvas 2D:** Cumplido. Se implementó el espacio de diseño con módulos de cajonera, modular, estante, base, divisor y cubierta, permitiendo arrastrar, mover y editar las dimensiones de cada módulo.
- **OE2 — Módulo de cubicación:** Cumplido. Se implementó el algoritmo de generación de piezas y cálculo de planchas, con un panel que lista las piezas por módulo junto al total de material requerido.
- **OE3 — Repositorio de muebles:** Cumplido. Se desarrolló la biblioteca de muebles que permite guardar, cargar y eliminar diseños previamente realizados.
- **OE4 — Biblioteca de materiales:** Cumplido. Se implementó el módulo de gestión de materiales con las categorías de material, accesorio, tapa-canto y cubierta, vinculados al cálculo de costos de la cotización.
- **OE5 — Cotizaciones:** Cumplido. Se implementó la generación de cotizaciones a partir del diseño, con previsualización, exportación a PDF y envío por correo electrónico.

- **OE6 — Reducción de tiempo:** A evaluar con el mueblista. La automatización de la cubicación y el cálculo de materiales elimina las horas de cálculo manual, por lo que se espera superar la reducción del 15 % propuesta.
- **OE7 — Estadísticas:** Cumplido. Se implementó el panel de estadísticas con KPIs de diseños, cotizaciones, monto acumulado y precio promedio, además de gráficos filtrables por año.
- **OE8 — Evaluación de usabilidad:** Se cuenta con el siguiente resultado en la encuesta SUS aplicada al mueblista:

1. Creo que me gustaría utilizar este sistema con frecuencia = 5
2. Encontré el sistema innecesariamente complejo = 2
3. Pensé que el sistema era fácil de usar = 4
4. Creo que necesitaría el apoyo de un técnico para poder utilizar este sistema = 1
5. Encontré que las diversas funciones de este sistema estaban bien integradas = 3
6. Pensé que había demasiada inconsistencia en este sistema = 1
7. Me imagino que la mayoría de la gente aprendería a utilizar este sistema muy rápidamente = 5
8. Encontré el sistema muy complicado de usar = 1
9. Me sentí muy seguro usando el sistema = 4
10. Necesitaba aprender muchas cosas antes de empezar con este sistema = 2

Suma de ítems impares: $(5 + 4 + 3 + 5 + 4) - 5 = 16$

Suma de ítems pares: $25 - (2 + 1 + 1 + 1 + 2) = 18$

Puntaje SUS: $(16 + 18) \times 2,5 = 85$

El puntaje obtenido de 85 puntos se ubica en la categoría *Acceptable* de la escala SUS (mayor a 68), lo que valida que la interfaz es intuitiva y adecuada para el perfil del mueblista artesanal, cumpliendo así el OE8.

5.2. Reflexión Final del Proyecto

Este proyecto fue un desafío que puso a prueba lo aprendido durante el desarrollo de la carrera, desde el planteamiento del problema, el cómo solucionarlo y el método para hacerlo, además de toda la aplicación de conocimientos técnicos que van de la mano con el desarrollo. Trabajar con la metodología PXP permitió avanzar de forma ordenada e iterativa, identificando en cada retrospectiva los ajustes necesarios para la siguiente iteración. El resultado final es una aplicación web funcional que cubre el flujo completo de trabajo del mueblista artesanal, desde el diseño hasta la entrega de la cotización al cliente.

5.3. Trabajo Futuro

Como trabajo futuro queda pendiente:

- Mejorar el comportamiento de la detección de colisiones entre figuras del canvas, especialmente para módulos como puertas y cubiertas que tienen lógica de posicionamiento distinta.
- Explorar un algoritmo de optimización de corte más avanzado (por ejemplo, un algoritmo de bin packing 2D) para reducir el desperdicio de material en el cálculo de planchas.
- Mejorar la interfaz visual de la aplicación con iconografía más representativa del rubro.
- Implementar la recuperación de contraseña por correo electrónico para el mueblista.
- Evaluar la escalabilidad del sistema para soportar múltiples usuarios de forma simultánea.

6. Carta Gantt

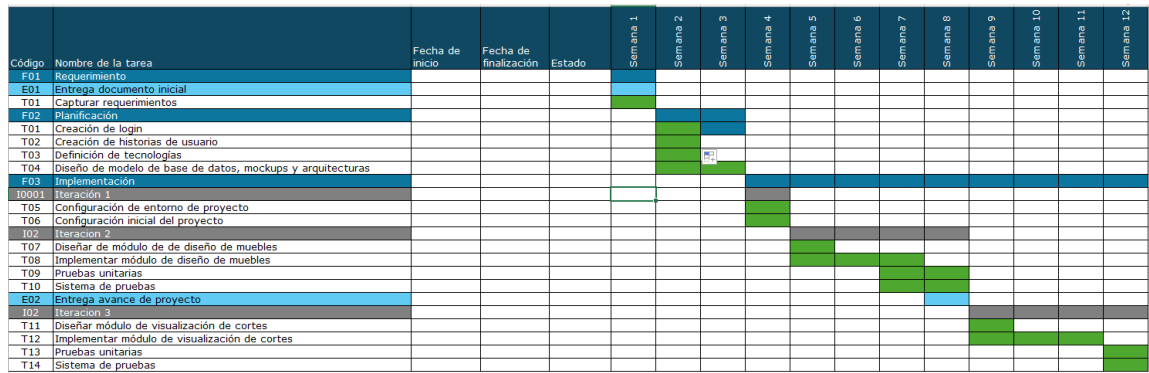


Figura 6.1: carta Gantt

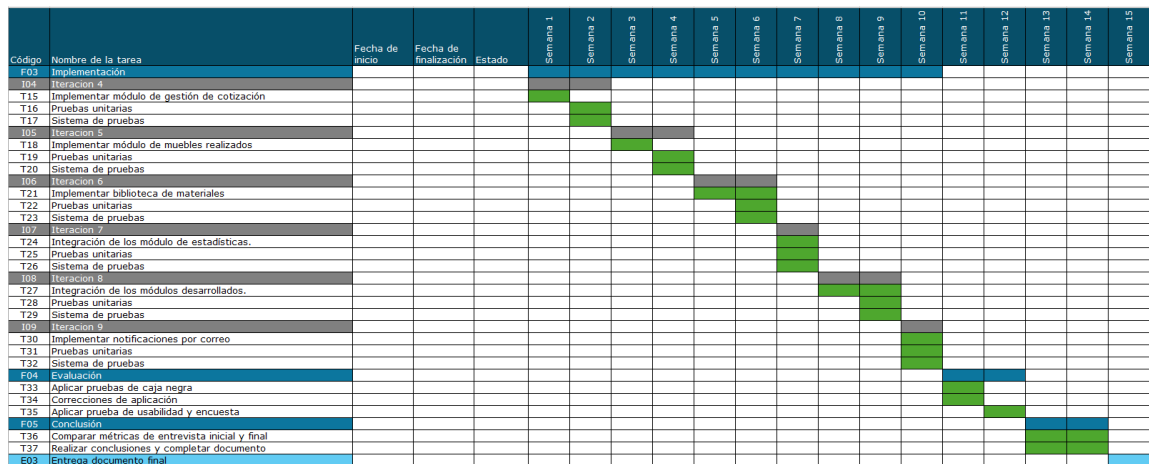


Figura 6.2: carta Gantt